BỘ CÔNG THƯƠNG

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THỰC PHẨM TP. HCM

KHOA: CÔNG NGHỆ THÔNG TIN

-----o0o----

ĐỒ ÁN MÔN HỌC

TÊN ĐỀ TÀI 19:

NGHIÊN CỨU CÁC THUẬT TOÁN MÁY HỌC ỨNG DỤNG TRONG PHÁT HIỆN TẤN CÔNG DDOS

Nhóm: 1: Giáp Đức Tài - 2033192280 2: Trần Minh Trí - 2033190135	**Giảng viên hướng dẫn:** **Ths. Trần Đắc Tốt**

Thành phố Hồ Chí Minh, tháng 7 năm 2022

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THỰC PHẨM TP. HỒ CHÍ MINH
KHOA: CÔNG NGHỆ THÔNG TIN
-----o0o----

ĐỒ ÁN MÔN HỌC

TÊN ĐỀ TÀI 19:

NGHIÊN CỨU CÁC THUẬT TOÁN MÁY HỌC ỨNG DỤNG TRONG PHÁT HIỆN TẤN CÔNG DDOS

Nhóm: 1: Giáp Đức Tài - 2033192280 2: Trần Minh Trí - 2033190135	**Giảng viên hướng dẫn:** **Ths. Trần Đắc Tốt**

Thành phố Hồ Chí Minh, tháng 7 năm 2022

LỜI CAM ĐOAN

Em/ chúng em xin cam đoan đề tài do cá nhân/nhóm em nghiên cứu và thực hiện.

Em/ chúng em đã kiểm tra dữ liệu theo quy định hiện hành.

Kết quả bài làm của đề tài là trung thực và không sao chép từ bất kỳ bài tập của nhóm khác.

Các tài liệu được sử dụng trong tiểu luận có nguồn gốc, xuất xứ rõ ràng.

Tp. Hồ Chí Minh, ngày 12 tháng 7 năm 2022

Sinh viên thực hiện Sinh viên thực hiện

LỜI CẢM ƠN

Lời đầu tiên chúng em xin gửi lời cảm ơn chân thành đến quý thầy, cô giáo trong khoa Công nghệ Thông Tin trường Đại học Công nghiệp thực phẩm Tp Hồ Chí Minh đã truyền đạt kiến thức và kinh nghiệm quý báu cho chúng em trong suốt quá trình học tập và rèn luyện .

Đặc biệt chúng em xin gởi lời cảm ơn đến thầy Trần Đắc Tốt, người đã trực tiếp hướng dẫn và tận tình giúp đỡ chúng em trong suốt thời gian học môn "Đồ án môn học" cũng như trong thời gian chúng em thực hiện thực bài tập kết thúc môn học.

Mặc dù đã cố gắng trong quá trình làm bài nhưng không thể tránh khỏi những sai sót. Chúng em mong nhận được sự góp ý, nhận xét của quý thầy cô và các bạn về nội dung cũng như hình thức trình bày để bài báo cáo được hoàn thiện.

Chúng em xin chân thành cảm ơn thầy cô và các bạn đã xem.

Tp. Hồ Chí Minh, ngày 12 tháng 7 năm 2022

Sinh viên thực hiện Sinh viên thực hiện

MỤC LỤC

PHẦN MỞ ĐẦU **1**
CHƯƠNG 1: TỔNG QUAN VỀ DDOS VÀ BOTNET **2**
1.1 Giới thiệu về Botnet **2**
1.2 Giới thiệu mô hình của Botnet **2**
1.2.1 Mô hình Star 3
1.2.2 Mô hình Random 4
1.2.3 Mô hình Multi - Server 5
1.2.4 Mô hình Hierachical 6
1.3 Phân loại botnet **7**
1.3.1 Phân loại theo chức năng hoặc mục đích 7
1.3.2 Phân loại botnet dựa trên mô hình của chúng 9
1.3.3 Phân loại theo nền tảng hệ điều hành để tấn công mục tiêu 10
1.3.4 Phân loại dựa theo phần mềm độc hại được cài đặt trên botnet 10
1.4 Cách Botnet lây nhiễm **11**
1.4.1 Đính kèm mã độc lừa người dùng chạy chúng 11
1.4.2 Tấn công vào các lỗ hổng chưa được vá (Zero-day) 12
1.4.3 Backdoor nạn nhân bằng Trojan 12
1.4.4 Khả năng lây nhiễm của botnet 12
1.5 Cơ chế hoạt động của botnet **15**
1.6 Điều khiển botnet **17**
1.7 Giải pháp phòng chống DDoS **21**
CHƯƠNG 2: NGHIÊN CỨU CÁC THUẬT TOÁN MÁY HỌC TRONG PHÁT HIỆN TẤN CÔNG DDOS **22**
2.1 Học có giám sát (Supervised Learining) **22**
2.2 Các thuật toán máy học phân loại trong học có giám sát **23**
2.2.1 Decision Tree 23
2.2.2 Random Forest 24
2.2.3 Navie Bayes 25
2.2.4 K-nearest neighbors (KNN) 27
2.2.5 Logistic Regression 28
2.2.6 XGBoost 29

2.3 Các kỹ thuật sử dụng trong tiền xử lý dữ liệu 29
2.3.1 Kỹ thuật xử lý các giá trị rỗng trong bộ dữ liệu 29
2.3.2 Xử lý các biến giá trị phân loại.....31
CHƯƠNG 3: XÂY DỰNG MÔ HÌNH MÁY HỌC VÀ THỰC NGHIỆM.....35
3.1 Xây dựng mô hình máy học.....35
3.2 Bộ dữ liệu CTU-13 35
3.3 Giải thích thực nghiệm triển khai mô hình máy học.....38
3.3.1 Tiền xử lý dữ liệu 38
3.3.2 Mô hình kịch bản sử dụng để thực nghiệm.....43
3.4 Tiến hành đánh giá kết quả thực nghiệm 44
KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN.....46
TÀI LIỆU THAM KHẢO 47

DANH MỤC HÌNH ẢNH

Hình 1.1: Sơ đồ mô hình star của botnet 3
Hình 1.2: Sơ đồ mô hình Random.....4
Hình 1.3: Sơ đồ mô hình Multi-Server C&C.....5
Hình 1.4: Sơ đồ mô hình Hierachical 6
Hình 1.5: Giao diện cấu hình thông tin của Agobot[10] 18
Hình 1.6: Botmaster kết nối IRC server và liên kết đến kênh dẫn để ra lệnh [10]...19
Hình 1.7: Ta truy cập vào tài khoản botmaster đã tạo [10] 19
Hình 1.8: Botmaster đưa ra lệnh đầu tiên cho bot con đầu tiên [10] 20
Hình 1.9: Botmaster đưa ra yêu cầu cho bot con thứ 2 của mình [10].....20
Hình 2.1: Ví dụ cây quyết định 23
Hình 2.2: Ví dụ về thuật toán Random Forest 24
Hình 2.3: Điểm dữ liệu mới cần phân loại.....27
Hình 2.4: Điểm dữ liệu mới cần phân biệt thuộc kiểu B.....28
Hình 2.5: Ví dụ về Logistic Regression.....28
Hình 2.6: Ví dụ về mã hóa với giá trị số.....31
Hình 2.7: Ví dụ về mã hóa one-hot encoding 31
Hình 3.1: Sơ đồ xây dựng các mô hình máy học 35
Hình 3.2: Cột tính năng label ở dữ liệu thô 40

Hình 3.3: Giá trị nằm ở cột Dir và Proto ở dữ liệu thô.....41
Hình 3.4: Mô hình kịch bản thực nghiệm[18] 43

DANH MỤC BẢNG BIỂU

Bảng 1.1: Danh sách mạng botnet tổng hợp được từ năm 2000 – 2021 12
Bảng 1.2: Danh sách 1 các cổng gắn với lỗ hổng dịch vụ [10] 16
Bảng 1.3: Danh sách lệnh mà Agobot thực thi được [10].....21
Bảng 2.1: Dữ liệu cho Ví dụ của Navie Bayes [13].....25
Bảng 3.1: Table 2 - Characteristic of the botnet scenarios [22].....36
Bảng 3.2: Lưu lượng dữ liệu được capture lại của mỗi kịch bản [22].....37
Bảng 3.3: Phân bố các label trong NetFlows của từng kịch bản [22].....37
Bảng 3.4: Mô tả chi tiết các cột dữ liệu.....38
Bảng 3.5: Trường hợp độ chính xác với mã hóa với giá trị số 44
Bảng 3.6: Trường hợp kỹ thuật lựa chọn tính năng Chi-Squared.....45
Bảng 3.7: Trường hợp sử dụng kỹ thuật lựa chọn tính năng ANOVA 45

DANH MỤC KÝ HIỆU VÀ TÓM TẮT

Viết tắt	Tiếng Anh	Tiếng việt
DDoS	Distributed Denial Of Service	Phân tán từ chối dịch vụ
CTU-13	CTU-13	Bộ dữ liệu CTU-13
UDP	User datagram protocol	Giao thức datagram người dùng
ICMP	Internet Control Message Protocol	Giao thức thông báo kiểm soát Internet
IP	Internet Protocol Address	Địa chỉ giao thức Internet
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận
IRC	Internet Relay Chat	Trò chuyện chuyển tiếp Internet
C&C	Command – and - Control	Ra lệnh và kiểm soát
ANOVA	Analysis of Variance	Phân tích phương sai

PHẦN MỞ ĐẦU

Trong thời đại công nghiệp 4.0 hiện nay, thì vai trò của Công nghệ Thông tin nói chung và Internet nói riêng ngày càng vô cùng quan trọng. Điều này kéo theo đa phần các doanh nghiệp, công ty hay các tổ chức đều phải phụ thuộc vào máy vi tính. Đồng thời với các lợi ích từ mạng máy tính thì cũng có các "tác dụng phụ" như các trang web lừa đảo, email độc hại, virus phá hoại, vv. DDoS - tấn công từ chối dịch vụ phân tán là 1 kiểu tấn công phổ biến làm tắc nghẽn máy chủ của đang hoạt động bằng cách yêu cầu quyền truy cập vào máy chủ với số lượng lớn, để làm được như vậy thì cần 1 đội máy tính lớn truy cập. Botnet ra đời để giải quyết điều này 1 đội máy tính zombie mạng lưới được chỉ huy để gửi các gói tin truy cập vào máy chủ.

Để hiểu rõ hơn về các tính năng cũng như cách thức hoạt động của Botnet này thì trong bài báo cáo này chúng em sẽ mang đến cái nhìn tổng quan hơn cũng như tác dụng rõ ràng về Botnet cũng như các thuật toán máy học phòng chống Botnet.

Từ những lí do trên, chúng em quyết định lựa chọn đề tài cho cuối kì là "nghiên cứu các thuật toán máy học ứng dụng trong phát hiện tấn công DDos".

Phạm vi nghiên cứu của đề tài lần này của em/chúng em là về DDos Botnet.

Phương pháp nghiên cứu đề tài của em/chúng em là tham khảo tài liệu, suy nghĩ chủ động và thực hiện được nội dung đã tham khảo và từ đó cải thiện nội dung của đề tài.

Chúng em xin chân thành cảm ơn thầy Trần Đắc Tốt, người đã dẫn trong quá trình thực hiện đề tài.

CHƯƠNG 1: TỔNG QUAN VỀ DDOS VÀ BOTNET

1.1 Giới thiệu về Botnet

Botnet là sự kết hợp giữa hai từ đó là “bot” và “network” đây là một hình thức botnet điều hành 1 đội quân máy zombie vì các máy thật đang được điều khiển bởi người khác chứ không phải là chủ sở hữu của chúng. Mỗi máy tính trên mạng hoạt động như một “ bot”, và được kẻ xấu kiểm soát để làm nhiều mục đích khác nhau như phát tán ransomware sang máy tính xách tay, điện thoại, máy tính bảng, vv…[1]. Chúng có thể bị phát hiện hoặc không bị phát hiện và tôi hay bạn đôi khi khó lòng biết được chúng ta đã bị trở thành 1 phần của Bonet hay chưa?. Tại vì quy mô của chúng có thể lớn tới mức một mạng của botnet (chứa hàng triệu botnet) cho phép kẻ tấn công tấn công trên mục tiêu lớn.

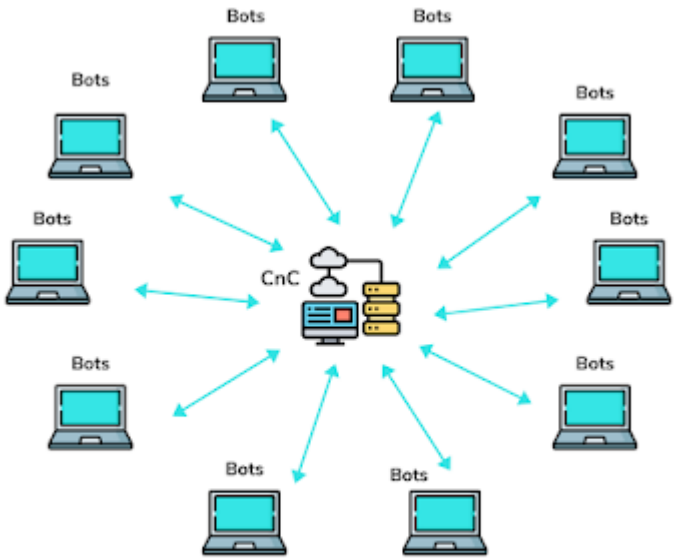
Nguồn gốc của bot net chủ yếu sử dụng và phục vụ trong các kênh Internet Relay Chat (IRC) nhưng kẻ xấu đã lợi dụng chúng, chúng khai thác các lổ hổng mạng trong IRC và các botnet được phát triển. Điều này được cố ý tạo ra như các hành động ghi nhớ hành động ấn phím, ăn cặp mật khẩu, v. v…[1].

1.2 Giới thiệu mô hình của Botnet

Cấu trúc botnet thường có 1 trong 4 dạng mô hình: Star (tập trung máy chủ), Random, Hierchical (phân cấp), multi - server (nhiều máy chủ). Theo thời gian ngày càng phát triển thì mô hình botnet được phát triển với nổ lực là tránh bị phát hiện và gián đoạn. Theo truyền thống thì cấu trúc mô hình botnet thường được xây dựng theo mô hình máy chủ tập trung, máy khách giao tiếp thông qua các máy chủ hiện có . Điều này cho phép bot herder (người điều khiển bot) thực hiện tất cả mọi thứ từ vị trí từ xa. Nhiều mạng botnet hiện này dựa vào mô hình random hiện có để giao tiếp. Các chương trình này thực hiện hành động tương tự mô hình star nhưng chúng không cần máy chủ trung tâm để liên kết với nhau. Để khắc phục các hạn chế của mô hình tập trung thì ta có mô hình hierchical (phân cấp) và mô hình multi - server (nhiều máy chủ) giúp ẩn giấu cũng như hạn chế việc server bị sập làm ảnh hưởng tới mạng botnet.

1.2.1 Mô hình Star

Trong cấu trúc client-server, một mạng cơ bản được thiết lập với máy chủ, hoạt động như một botmaster. Botmaster kiểm soát việc truyền thông tin mỗi client để thiết lập lệnh và điều khiển các thiết bị khách. Mô hình này hoạt động với sự trợ giúp của các phần mềm đặc biệt và cho phép botmaster duy trì quyền kiểm soát. Mô hình này có vài nhược điểm như nó có thể định vị được botmaster thông qua máy client và chỉ có 1 điểm kiểm soát các botnet con. Trong mô hình này nếu botmaster – máy server bị đánh sập thì botnet sẽ "chết"[5].



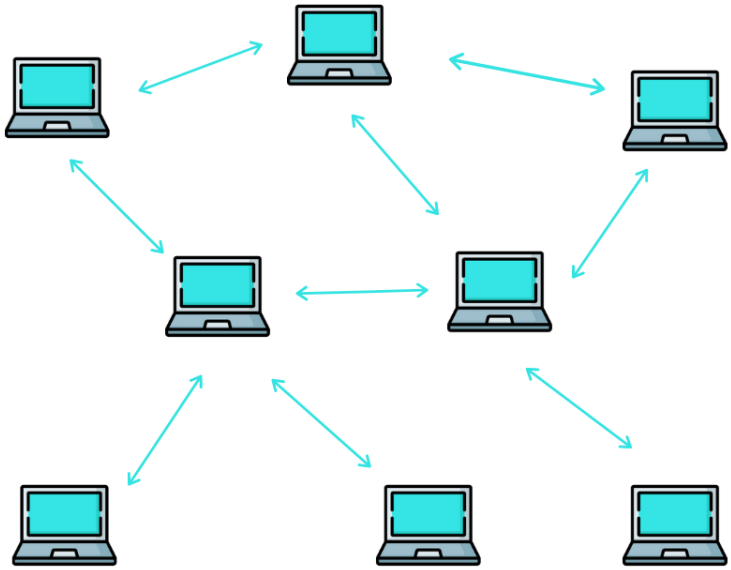
Hình 1.1: Sơ đồ mô hình star của botnet

Ưu điểm: Tốc độ kiểm soát cực kì nhanh chóng nhiều máy bot giao tiếp với máy chủ nên luôn luôn giao tiếp 1 cách trực tiếp hầu như độ trễ lệnh là không có.

Nhược điểm: Có thể bị giám sát và phát hiện máy chủ. Nếu máy chủ bị đánh sập hoặc bị chặn thì mạng botnet sẽ chết và ngừng hoạt động.

1.2.2 Mô hình Random

Để khắc phục nhược điểm của việc phụ thuộc vào một máy chủ tập trung thì các mạng botnet đã phát triển ra mô hình random được kết nối dưới dạng ngang hàng [4]. Cấu trúc mô hình này chặt chẽ hơn do không dồn vào một máy trung tâm điều khiển mô hình này không cần có cơ sở thiết bị quá tốt vẫn có thể dễ dàng triển khai. Nhưng mô hình này dễ bị giám sát và xác định được các máy bot khác thông qua việc giám sát một máy bot giao tiếp với máy khác và độ trễ lệnh là thứ không thể tránh khỏi với mô hình này.



Hình 1.2: Sơ đồ mô hình Random

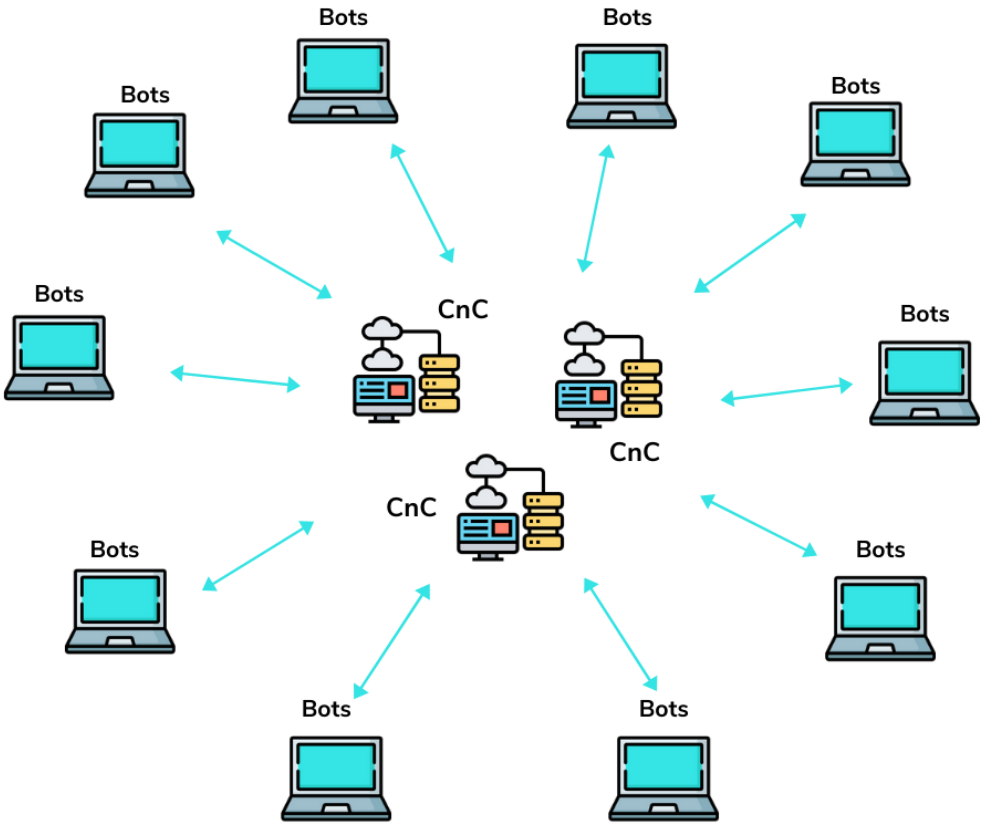
Ưu điểm: Có khả năng phục hồi cao do chỉ cần thêm botnet client vào là có thể hồi phục được. Không cần cơ sở hạ tầng quá lớn để có máy chủ C&C giúp tiết kiệm chi phí vận hành.

Nhược điểm: Có độ trễ lệnh khi phải truyền qua nhiều máy botnet khác nhau nếu mô hình quá nhiều cụm bot thì sẽ phải rất lâu thông tin mới đi hết được mô hình. Truy tìm botnet rất dễ dàng, khi có kẻ khác tấn công vào truy cập vào 1 máy bot có thể truy xuất được toàn bộ thông tin của các máy bot khác trong mạng vì bảo mật kém hiệu quả hơn máy chủ C&C.

1.2.3 Mô hình Multi - Server

Mô hình này là mô hình mở động dựa trên mô hình tập trung client-server, trong đó bao gồm nhiều máy chủ C&C được sử dụng . Nhiều máy chủ sẽ giao tiếp với nhau khi chúng quản lí mạng botnet. Nếu một máy chủ nào đó trong mô hình bị lỗi thì mạng botnet vẫn có tiếp tục hoạt động vào các máy chủ C&C còn lại [2].

Tuy nhiên botmaster này cần phải có kế hoạch chi tiết khi xây dựng mô hình này. Một điều lợi ích là các bot giống nhau có thể sử dụng cho cả mô hình tập trung 1 máy chủ C&C hoặc nhiều máy chủ C&C topology [2].



Hình 1.3: Sơ đồ mô hình Multi-Server C&C

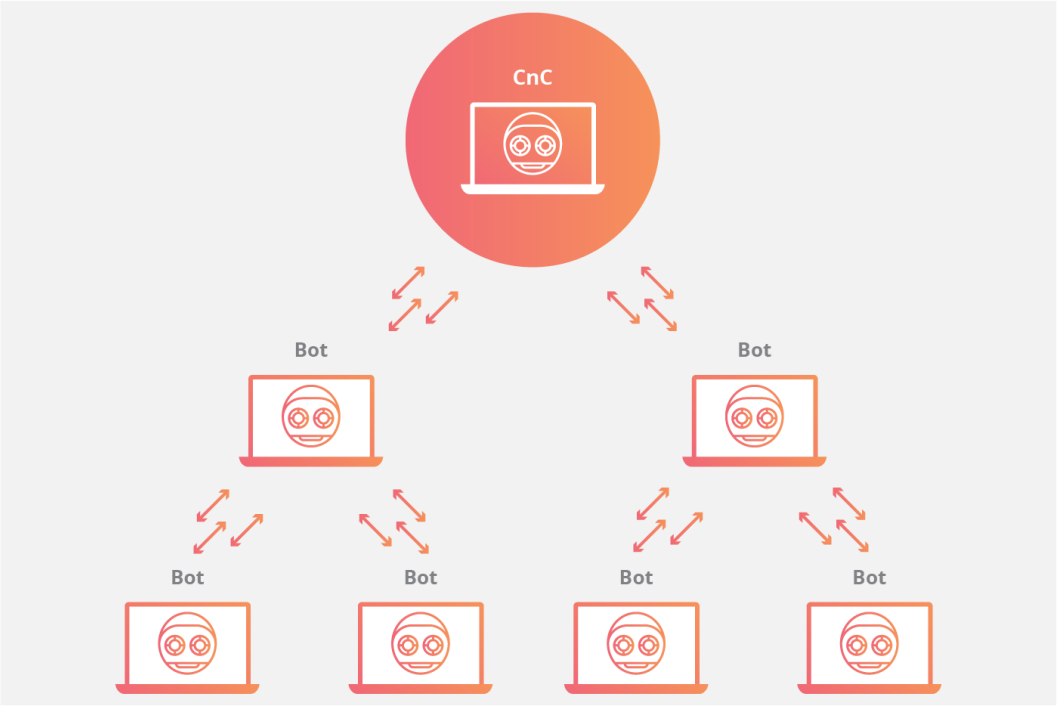
Ưu điểm: Không có lỗi nào nếu một máy chủ trong mô hình bị sập mạng botnet vẫn có thể duy trì bình thường và ổn định nhờ vào các máy chủ còn lại trong mạng. Tối ưu hóa vị trí khi đặt nhiều máy chủ ở nhiều nơi khác nhau sẽ tăng tốc độ trao đổi thông tin của các bot trong mạng botnet.

Nhược điểm: Cần có sự chuẩn bị kĩ lưỡng khi tạo mô hình và hiểu biết về cơ sở hạ tầng của mô hình này.

1.2.4 Mô hình Hierachical

Các bot thế hệ đầu trong mô hình này có khả năng được ủy quyền các lệnh C&C cho các botnet thế hệ sau được thêm vào mạng. Tuy nhiên, các lệnh hướng dẫn được cập nhật thường gặp phải độ trễ khiến botmaster khó sử dụng cho các hoạt động yêu cầu thời gian thực [2].

Hierachical botnet (botnet phân cấp) có nghĩa là không có botnet nào nhận biết được vị trí toàn bộ của mạng botnet. Cấu hình kiểu này gây ra khá nhiều thách thức, khó khăn trong việc bảo mật mạng cũng như mô tả và ước tính kích thước của mạng botnet [2].



Hình 1.4: Sơ đồ mô hình Hierachical

Ưu điểm của mô hình này: che giấu và ẩn được mạng botnet giúp ẩn được máy chủ thật sự trong mô hình này. Dễ bán lại hoặc đem đi cho thuê, có thể dễ dàng cắt 1 phần botnet ra đem chúng đi cho thuê hoặc bán lại nếu muốn.

Nhược điểm của mô hình: độ trễ về các lệnh sẽ bị khá nhiều vì đi qua khá là nhiều lớp phân cấp mạng botnet có thể có độ trễ cao với các lệnh cập nhật được nhận của bot thế hệ sau. Điều này dẫn đến sự không đồng bộ làm giảm chất lượng của mô hình mạng.

1.3 Phân loại botnet

Việc phân loại botnet không hề dễ dàng. Chúng có thể được phân loại theo một số tiêu chí khác nhau như:

- Chức năng hoặc mục đích botnet.
- Kiến trúc botnet hoặc xây dựng.
- Các nền tảng mà họ đang nhắm mục tiêu (Windows, Linux, vv...).
- Dựa trên loại phần mềm độc hại được cài đặt của botnet.
- Dựa trên cách họ lây nhiễm vào máy của nạn nhân (ví dụ như sử dụng bộ khai thác, thông qua lỗ hổng trình duyệt, thư rác email, vv...) [6].

1.3.1 Phân loại theo chức năng hoặc mục đích

Hầu hết các botnet được thiết kế để dễ dàng kiểm soát và quản lý. Chúng cho phép kiểm soát một số lượng lớn các máy tính bị nhiễm từ một máy tính duy nhất. Các botnet này cung cấp một loạt các chức năng, chẳng hạn như tiếp quản máy của nạn nhân, đánh cắp dữ liệu trên đó, theo dõi hoạt động của người dùng bằng cách ghi lại các tổ hợp phím hoặc chụp ảnh màn hình, gửi tin nhắn rác cho người nhận vô tội và cũng thực hiện Tấn công DDoS. Dựa trên chức năng của chúng, chúng ta có thể phân biệt [6].

Botnet chỉ kiểm soát một máy tính (zombie) dựa trên địa chỉ IP của nó. Loại botnet này được kiểm soát thông qua các kênh IRC bằng cách sử dụng một ứng dụng phần mềm đặc biệt. Bot là các chương trình được viết đặc biệt có thể được lập trình để thực hiện các hành động vào những thời điểm nhất định hoặc trong một số tình huống nhất định. Loại botnet này là dễ tạo và hoạt động nhất vì nó không yêu cầu bất kỳ cơ sở hạ tầng hoặc máy chủ C&C nào; do đó, các botnet này thường được tội phạm mạng sử dụng cho các hoạt động bất hợp pháp như gửi tin nhắn rác, mở cửa hậu trên máy của nạn nhân và thực hiện các cuộc tấn công DdoS [6].

Botnet kiểm soát một số lượng lớn máy tính (zombie) dựa trên địa chỉ IP của chúng bằng cách sử dụng Internet Relay Chat (IRC). Mạng IRC đã trở thành cách phổ biến nhất để các nhà khai thác botmaster kiểm soát bot của họ. Với các botnet dựa trên IRC, những kẻ tấn công có thể tuyển dụng các hệ thống máy tính bị xâm nhập từ khắp nơi trên thế giới vào một nhóm duy nhất mà không gặp nhiều khó khăn.

Để thiết lập một botnet dựa trên IRC, kẻ tấn công trước tiên xác định các máy dễ bị tổn thương bằng các phương pháp như:

- Quét cổng
- Quét lỗ hổng
- Khai thác quét

Kẻ tấn công sau đó sử dụng các công cụ khác nhau để xâm nhập hệ thống (điều này thường bao gồm cài đặt Trojan hoặc Backdoor trên máy tính của nạn nhân thông qua một bộ khai thác). Những loại botnet này được phân phối trên toàn thế giới và có thể thực hiện các nhiệm vụ cho tội phạm mạng bất cứ lúc nào. Các botnet này thường bao gồm các Trojan tùy chỉnh được thiết kế đặc biệt để đánh cắp tiền từ người dùng trên toàn thế giới thông qua các kế hoạch gian lận.

Botnet kiểm soát một số lượng lớn máy tính (zombie) dựa trên địa chỉ IP của chúng bằng cách sử dụng các công cụ quản trị từ xa (RATs) được cài đặt trên mỗi máy tính để cho phép kẻ tấn công chiếm quyền kiểm soát máy theo ý muốn. RAT là một chương trình phần mềm được sử dụng để kiểm soát hoàn toàn một hoặc nhiều máy tính từ một vị trí từ xa. RAT là các chương trình độc hại có thể được cài đặt trên máy tính của nạn nhân mà không có kiến thức và sự đồng ý của họ, thường sử dụng:

- Trang web cập nhật phần mềm giả mạo.
- Trojans.
- Phần mềm gián điệp.
- Keyloggers.
- Phần mềm độc hại khác.

Khi kẻ tấn công cài đặt RAT trên (các) máy mục tiêu, Hắn có toàn quyền truy cập máy tính và đánh cắp dữ liệu từ ổ cứng, tải lên mã độc theo dõi hoạt động của người dùng và làm nhiều hơn nữa. Nhà điều hành botmaster kiểm soát các hệ thống bị xâm nhập này bằng cách cài đặt các mô-đun phần mềm bổ sung vào chúng từ xa thông qua mạng Internet Relay Chat (IRC) [6].

Botnet chạy các cuộc tấn công từ chối dịch vụ phân tán (DDoS) dựa trên địa chỉ IP của chúng. Các botnet này được kiểm soát thông qua IRC bằng cách sử dụng các lệnh đặc biệt để khởi chạy Tấn công DDoS chống lại các mục tiêu của họ. Khi

một trong những bot này nhận được lệnh từ kẻ tấn công, nó sẽ phát động một cuộc tấn công vào máy chủ web, mạng hoặc các hệ thống máy tính khác của nạn nhân. Loại botnet này rất phổ biến trong số các tội phạm mạng cho thuê tài nguyên của họ thông qua các diễn đàn và dịch vụ trực tuyến khác nhau cho bất kỳ ai muốn thực hiện một cuộc tấn công DDoS chống lại một bên khác mà không tiết lộ danh tính của họ [5].

1.3.2 Phân loại botnet dựa trên mô hình của chúng

Mô hình máy khách-máy chủ. Botnet máy chủ khách hàng là loại botnet phổ biến nhất. Nó sử dụng một máy chủ C&C trung tâm, nhận và gửi lệnh đến các máy bị nhiễm theo cách này hay cách khác. Tin nhắn lệnh được gửi dưới dạng lệnh có thể được thực thi trên máy tính bị đánh cắp.

Mô hình ngang hàng. Trong các botnet ngang hàng (P2P), mọi nút đều có thể hoạt động như một máy khách và máy chủ; chúng kết nối trực tiếp với nhau bằng các giao thức IRC hoặc HTTP. Kẻ tấn công không cần phải thiết lập bất kỳ máy chủ đặc biệt nào cho loại cấu trúc mạng này, nhưng anh ta vẫn có quyền kiểm soát các hoạt động độc hại được thực hiện bởi các thiết bị bị nhiễm mà không có bất kỳ trung gian nào.

Telnet. Botnet Telnet sử dụng một giao thức botnet C&C đơn giản, trong đó các bot kết nối với máy chủ lệnh chính để lưu trữ botnet. Bot được thêm vào botnet bằng cách sử dụng một tập lệnh quét, chạy trên một máy chủ bên ngoài và quét phạm vi IP cho telnet và máy chủ SSH đăng nhập mặc định. Khi đăng nhập được tìm thấy, máy chủ quét có thể lây nhiễm nó thông qua SSH bằng phần mềm độc hại, ping máy chủ điều khiển.

IRC. Các mạng IRC sử dụng các phương pháp truyền thông băng thông đơn giản, thấp, làm cho chúng được sử dụng rộng rãi để lưu trữ botnet. Chúng có xu hướng tương đối đơn giản trong xây dựng và đã được sử dụng với thành công vừa phải để điều phối các cuộc tấn công DDoS và các chiến dịch spam trong khi có thể liên tục chuyển kênh để tránh bị gỡ xuống.

Botnet pha trộn các mô hình với nhau. Phần lớn các botnet là sự pha trộn của hai mô hình đầu tiên được mô tả ở trên. Tuy nhiên, cũng có những botnet có các yếu tố từ cả ba hoặc bốn mô hình trong đó.

1.3.3 Phân loại theo nền tảng hệ điều hành để tấn công mục tiêu

Botnet dựa trên Windows. Do sự phổ biến của chúng, các hệ thống Microsoft Windows là một trong những nền tảng phổ biến nhất để lây nhiễm phần mềm độc hại và chạy các chương trình độc hại như bot. Ngoài ra, chúng rất phổ biến trong số các nhóm tội phạm mạng do dễ dàng truy cập và một số lượng lớn các lỗ hổng trong hệ thống này. Đây là lý do tại sao chạy các phiên bản lỗi thời của hệ điều hành này rất nguy hiểm.

Botnet dựa trên Linux. Botnet dựa trên Linux có hai mục tiêu chính: máy chủ và máy tính gia đình. Do sự phổ biến của chúng trong việc vận hành các trang web Internet, dịch vụ tải lên tệp và các dịch vụ khác, số lượng bot trên Linux đang tăng nhanh. Máy Linux cũng được sử dụng rộng rãi trong các thiết bị như bộ định tuyến gia đình và camera IP.

Botnet dựa trên Android. Botnet Android tương đối mới, nhưng theo thời gian, chúng có khả năng trở thành một mối đe dọa rất phổ biến. Các chương trình độc hại cho các thiết bị Android đã được phát hiện (Trojan-SMS. AndroidOS. Opfake) và nhiều chương trình khác chắc chắn sẽ xuất hiện khi sự phổ biến của hệ điều hành này tăng lên. [5]

1.3.4 Phân loại dựa theo phần mềm độc hại được cài đặt trên botnet

Botnet DDoS. Hàng trăm hoặc hàng ngàn máy tính bị nhiễm có thể được sử dụng để khởi động các cuộc tấn công DDoS mạnh mẽ chống lại một mục tiêu được chỉ định (xem sơ đồ bên dưới). Chủ sở hữu của các botnet này có thể cung cấp dịch vụ của họ cho thuê.

Botnet thăm dò mạng. Các máy bị nhiễm có một mục đích - quét Internet và tìm các máy tính dễ bị tổn thương khác để lây nhiễm chúng bằng phần mềm độc hại có khả năng biến chúng thành bot. Những loại bot thường nhắm vào các mục tiêu cụ thể (chẳng hạn như máy chủ), nơi họ đang cố gắng giành quyền kiểm soát hoàn toàn toàn bộ hệ thống máy tính, bao gồm tất cả dữ liệu, phần mềm và tài nguyên phần cứng của nó. Một botnet như vậy sẽ được coi là hình thành đầy đủ một khi nó có quyền truy cập vào thậm chí chỉ một máy chủ mà trên đó các tài nguyên phần mềm / phần cứng mạnh mẽ đang chạy.

Botnet backdoor. Máy tính bị nhiễm được sử dụng để lây nhiễm các máy tính khác và thêm chúng vào danh sách bot có thể được điều khiển bởi kẻ tấn công.

Botnet gửi thư rác. Những loại phần mềm độc hại này được thiết kế để gửi hàng triệu (hoặc thậm chí hàng tỷ) tin nhắn không mong muốn từ các máy tính bị nhiễm trên khắp thế giới đến các nạn nhân tiềm năng của chúng. Địa chỉ email được sử dụng bởi các botnet này có thể được thu thập trên các trang web công cộng hoặc thông qua các phương tiện khác (ví dụ, bằng cách lây nhiễm các máy khác). Các mạng bao gồm hoàn toàn các thiết bị bị xâm nhập như bộ định tuyến, camera IP, máy in, v. v. sử dụng các kỹ thuật tương tự khi gửi thư rác. [6]

1.4 Cách Botnet lây nhiễm

Botnet rất khó phát hiện. Họ chỉ sử dụng một lượng nhỏ sức mạnh tài nguyên của hệ thống để tránh làm gián đoạn các chức năng thiết bị bình thường và cảnh báo người dùng. Các botnet tiên tiến hơn thậm chí còn được thiết kế để cập nhật hành vi của chúng để ngăn chặn sự phát hiện của phần mềm an ninh mạng. Người dùng không biết rằng họ đang được kết nối thiết bị đang bị kiểm soát bởi tội phạm mạng. Điều tồi tệ hơn, thiết kế botnet tiếp tục phát triển, làm cho các phiên bản mới hơn khó tìm hơn. Đa số tất cả chúng đều được lây nhiễm qua các máy tính của nạn nhân theo các kiểu sau đây:

- Đính kèm những mã độc lừa người dùng chạy chúng
- Tấn công vào những lỗ hỗng chưa được vá.
- Backdoor nạn nhân bằng Trojan

1.4.1 Đính kèm mã độc lừa người dùng chạy chúng

Một số ví dụ về việc này sau đây:

Phishing: Lừa đảo câu dẫn dụ người dùng sử dụng ấn vào trang web có sẵn mã độc sẵn đã được cài đặt sẵn, đôi khi thuyết phục người dùng đưa những tài khoản và mật khẩu cá nhân như ngân hàng, facebook, zalo, vv…

Gửi đến các e-mail có chứa các tệp mã độc sẵn khi ấn vào thì mã độc được thực thi .

Spam tin nhắn qua e-mail hoặc qua các nền tảng mạng xã hội: Các tin nhắn với các nội dung khuôn mẫu ví dụ như: “ Bạn đã vi phạm nội dung tiêu chuẩn cộng động vui lòng vào link sau đây để giải quyết nếu không sẽ bị khóa tài khoản “ hoặc có thể đánh

vào các tâm lí sở thích như: “ Bạn đã nhận được các quà tặng như sau vui lòng truy cập vào để nhận thưởng”.

1.4.2 Tấn công vào các lỗ hổng chưa được vá (Zero-day)

Để có thể tấn công vào các lỗ hổng zero-day chưa được vá này thì các botnet client thường bao gồm các khả năng quét các lỗ hổng kiểm tra các cổng đang mở trên máy nạn nhân từ đó thu thập lại danh sách các lỗ hổng này và báo lại cho botnet server. Botnet quét các hệ thống máy chủ có một trong số các lỗ hổng bảo mật mà khi bị xâm nhập, nó cho phép điều khiển từ xa máy chủ dễ bị tấn công. Một phát triển khá là mới là việc sử dụng Google có thể tìm kiếm các hệ thống dễ bị tấn công điều khiến cho có rất nhiều lỗ hổng bảo mất được rao bán hằng ngày trên thị trường được các tay hacker mua lại là rất nhiều. Tích cực vá các lỗ hổng này cũng là 1 cách tốt để để phòng ngừa. Nếu các lỗ hổng này ở các giao thức mạng số giao mà ta thường không sử dụng, tường lửa dựa trên máy chủ có thể bảo vệ hoặc hệ thống phát hiện xâm nhập IPS/IDS sẽ cảnh báo lưu lượng truy cập bất thường từ đây.

1.4.3 Backdoor nạn nhân bằng Trojan

Một số mạng botnet có thể tự tìm kiếm các backdoor do các đoạn mã độc hại khác như để lại Trojan trên máy nạn nhân. Đơn giản hiệu quả dễ sử dụng là những gì trojan mang lại được và dễ dàng điều khiển máy nạn nhân 1 cách mà không ai có thể biết được. Chúng thường hay được đặt trong phần cấu hình mặc định khi tạo ra 1 mạng botnet.

1.4.4 Khả năng lây nhiễm của botnet

Bảng 1.1: Danh sách mạng botnet tổng hợp được từ năm 2000 – 2021

Tên Botnet	**Năm**	**Xấp xỉ số bot trong mạng**
EarthLink Spammer	2000	1, 250, 000
Coreflood	2001	2, 300, 000
Bagle	2004	230, 000
Rustock	2006	150, 000
Cutwail	2007	1, 500, 000
Akbot	2007	1, 300, 000
Srizbi	2007	450, 000

Bayrob	2007	400, 000
Storm	2007	250, 000
Mariposa	2008	12, 000, 000
Conficker	2008	10, 500, 000
Sality	2008	1, 000, 000
Kraken	2008	495, 000
Waledac	2008	80, 000
Asprox	2008	15, 000
BredoLab	2009	30, 000, 000
ZeroAccess	2009	9, 000, 000
Bamital	2009	1, 800, 000
Grum	2009	560, 000
Festi	2009	250, 000
TDL4	2010	4, 500, 000
Kelihos	2010	300, 000
LowSec	2010	11, 000
Gameover Zeus	2010	không biết
Ramnit	2011	3, 000, 000
Andromeda (Gamarue)	2011	không biết
Dridex	2011	không biết
Carna	2012	420, 000
Chameleon	2012	120, 000
3ve	2013	1, 500, 000
Necurs	2014	6, 000, 000
Semalt	2014	300, 000
Emotet	2014	không biết
Bashlite	2014	không biết
Mirai	2016	500, 000
TrickBot	2016	200, 000
Methbot	2016	100, 000
Retadup	2017	850, 000
Smominru (Hexmen, MyKings)	2017	525, 000

Hajime	2017	300, 000
WireX	2017	120, 000
Emotet	2021	2, 000, 000
Beastmode	2021	152, 000
Moobot	2021	Không biết

Một số thông tin về những mạng botnet phổ biến:

BredoLab - một botnet đã bị đóng cửa vào năm 2014, nó bị nghi ngờ đã được sử dụng cho Tấn công DDoS nhắm vào các trang web của công ty và các ngân hàng có trụ sở tại Hoa Kỳ. [7]

Mariposa - botnet Mariposa ban đầu, bị đóng cửa vào năm 2007 và được sử dụng để kiểm soát hàng trăm ngàn máy tính bị nhiễm trên toàn thế giới. [7]

ZeroAccess - một botnet kiếm được hơn 500. 000 đô la mỗi tháng cũng được cho là đứng đằng sau một tỷ lệ lớn tất cả các email spam được gửi trên toàn thế giới. [7]

Conficker - một botnet gây ra rất nhiều lo ngại do sử dụng công nghệ root-kit, và do đó cực kỳ khó phát hiện phần mềm chống vi-rút. Nó được cho là đã lây nhiễm hơn 9 triệu máy tính trong vài tuần đầu tiên sau khi nó bắt đầu lan rộng. [7]

TDL4 - một botnet được xây dựng trên mạng Tor để ngụy trang khỏi bị phát hiện trong khi những người tạo ra nó kiếm tiền cho thuê nó. Ước tính nó đã lây nhiễm tới 4, 5 triệu máy tính trên toàn thế giới. [7]

Ramnit - một botnet được thiết kế để đánh cắp thông tin ngân hàng và thẻ tín dụng thông tin từ các máy bị nhiễm bệnh. Người ta ước tính rằng nó đã chịu trách nhiệm cho việc đánh cắp hơn 1 tỷ đô la dữ liệu tài chính kể từ năm 2011. [7]

Coreflood - một botnet được xây dựng trên mô hình ngang hàng, nơi các máy tính bị nhiễm có thể kết nối với nhau bằng cách sử dụng một số giao thức truyền thông như IRC hoặc HTTP. [7]

Mirai - một botnet được coi là một trong những botnet nguy hiểm nhất từng được tạo ra. Nó bao gồm mã cho phép nó quét các thiết bị khác trên mạng và sau đó sử dụng các thiết bị đó để tấn công các trang web và máy chủ web trong DDoS cuộc tấn công. [8]

Cutwail - một botnet có khả năng gửi hơn 1, 5 tỷ email spam mỗi ngày, khiến nó trở thành phần mềm độc hại phổ biến nhất từng được tạo ra. [7]

Necurs- Một botnet Đã lây nhiễm hơn 9 triệu máy tính trên toàn thế giới. Quân sự hóa 5 đến 6 triệu điểm cuối zombie (zombie endpoints) Nó sử dụng Necurs DGA - thuật toán tạo tên miền của botnet, thành phần tạo ra tên miền ngẫu nhiên. Necurs cũng có thể chặn các bản cập nhật chống vi-rút trong các máy cũ hơn, dẫn đến một loạt các vấn đề knock-on Đối với các thiết bị sử dụng Windows 7 đã lỗi thời mà không có các biện pháp bảo vệ chống vi-rút được cập nhật, Necurs không chỉ làm tê liệt cơ chế bảo mật có thể dẫn đến việc loại bỏ Necurs khỏi thiết bị máy tính, nó có thể khiến các thiết bị máy tính của nạn nhân tiếp xúc với nhiều loại phần mềm độc hại khác. [9]

1.5 Cơ chế hoạt động của botnet

Hoạt động tấn công của botnet có thể chia thành 4 giai đoạn:

Tạo botnet

Cấu hình botnet

Tấn công vào nạn nhân

Điều khiển botnet lấy thông tin khi tấn công thành công

Giai đoạn **Tạo** phụ thuộc lớn vào kỹ năng và đòi hỏi của kẻ tấn công. Nếu là người bẻ khoá chuyên nghiệp, họ có thể cân nhắc giữa việc viết mã bot riêng hoặc đơn giản chỉ là mở rộng, tuỳ biến cái đã có. Lượng bot có sẵn là rất lớn và khả năng cấu hình cao. Một số còn cho phép thao tác dễ dàng hơn qua một giao diện đồ hoạ. Giai đoạn này không có gì khó khăn, thường dành cho những kẻ mới vào nghề [10].

Giai đoạn **Cấu hình** là cung cấp server IRC và kênh thông tin. Sau khi cài đặt lên một máy tính đã được kiểm soát, bot sẽ kết nối tới host được chọn. Đầu tiên kẻ tấn công nhập dữ liệu cần thiết vào để giới hạn quyền truy cập bot, bảo vệ an toàn cho kênh và cuối cùng cung cấp một danh sách người dùng được cấp phép (những người có thể điều khiển bot). Ở giai đoạn này, bot có thể được điều chỉnh sâu hơn, như định nghĩa phương thức tấn công và đích đến [10].

Giai đoạn **Tấn công** là sử dụng nhiều kỹ thuật khác nhau để phát tán bot, cả trực tiếp và gián tiếp. Hình thức trực tiếp có thể là khai thác lỗ hổng của hệ điều hành hoặc

dịch vụ. Còn gián tiếp thường là triển khai một số phần mềm khác phục vụ cho công việc đen tối, như sử dụng file HTML dị dạng để khai thác lỗ hổng Internet Explorer, sử dụng một số phần mềm độc hại khác phân phối qua các mạng ngang hàng hoặc qua trao đổi file DCC (Direct Client–to–Client) trên IRC. Tấn công trực tiếp thường được thực hiện tự động thông qua các sâu (worm). Tất cả công việc những sâu này phải làm là tìm kiếm mạng con trong hệ thống có lỗ hổng và chèn mã bot vào. Mỗi hệ thống bị xâm phạm sau đó sẽ tiếp tục thực hiện chương trình tấn công, cho phép kẻ tấn công ghi lại tài nguyên đã dùng trước đó và có được nhiều thời gian để tìm kiếm nạn nhân khác [10].

Bảng 1.2: Danh sách 1 các cổng gắn với lỗ hổng dịch vụ [10]

Cổng	Dịch vụ
42	WINS (Host Name Server)
80	HTTP (lỗ hổng IIS hay Apache)
135	RPC (Remote Procedure Call)
137	NetBIOS Name Service
139	NetBIOS Session Service
445	Microsoft–DS–Service
1025	Windows Messenger
1433	Microsoft–SQL–Server
2745	Bagle worm backdoor
3127	MyDoom worm backdoor
3306	MySQL UDF (User Definable Functions)
5000	UPnP (Universal Plug and Play)

Giai đoạn ***Điều khiển*** gồm một số hoạt động thực hiện sau khi bot đã được cài đặt lên máy đích trong một thư mục chọn. Để khởi động với Windows, bot update các khoá đăng ký, thông thường là HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\. Việc đầu tiên bot thực hiện sau khi được cài đặt thành công là kết nối tới một server IRC và liên kết với kênh điều khiển thông qua sử dụng một mật khẩu. Nickname trên IRC được tạo ngẫu nhiên. Sau đó, bot ở trạng thái sẵn sàng chờ lệnh từ ứng dụng chủ. Kẻ tấn công cũng phải sử dụng một mật khẩu để kết nối tới botnet.

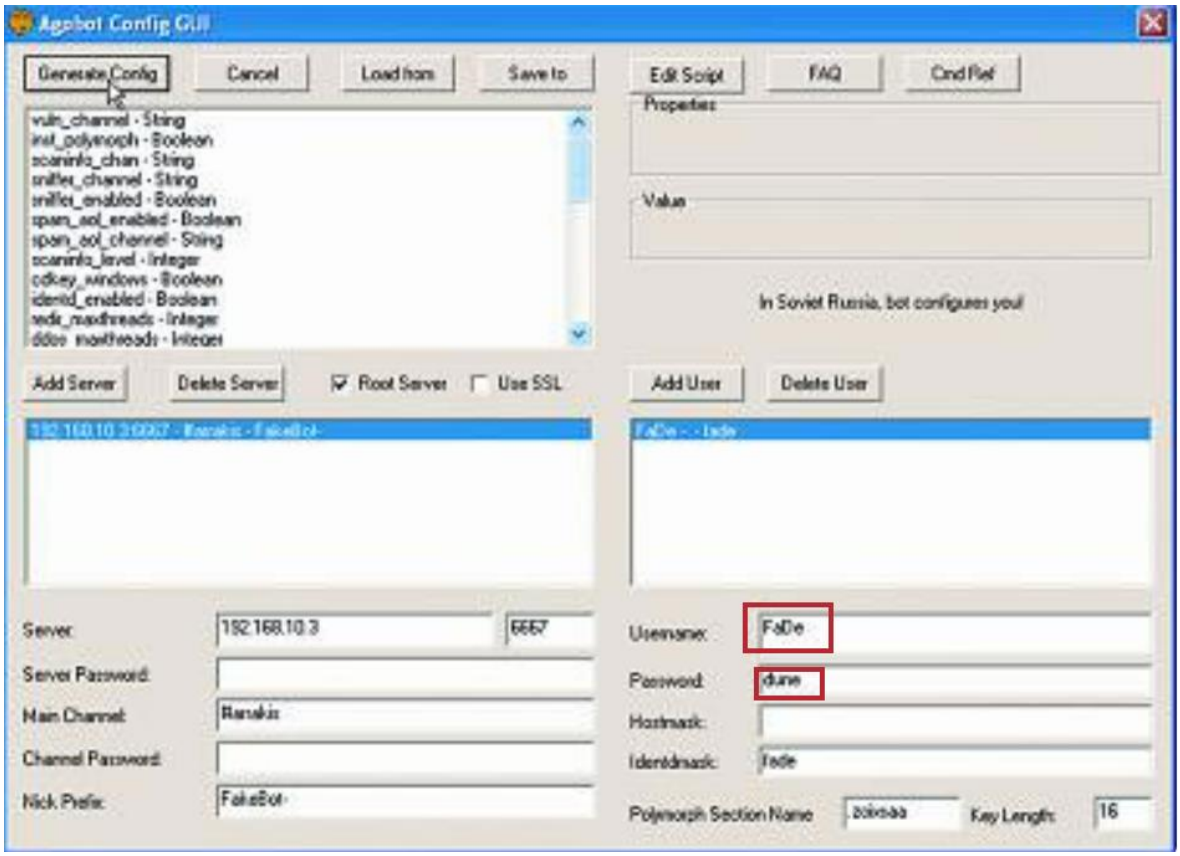
Điều này là cần thiết để không ai khác có thể sử dụng mạng botnet đã được cung cấp. [10]

1.6 Điều khiển botnet

Để hiểu rõ hơn cách botnet được điều khiển thì chúng ta cùng xem một ví dụ nạn nhân đã bị tấn công bằng sử dụng Agobot sau đây để hiểu một chương trình Comand and control được thực hiện như thế nào. Hai máy tính được sử dụng máy đầu tiên là chạy một server IRC dựa trên UnrealRCd 3. 2. 3 và hai hệ điều hành WindowXP SP1 ảo dựa trên Vmware Worksation (2 máy này sẽ là nạn nhân). Máy thứ 2 dành cho botmaster điều khiển botnet qua Irssi nó là một IRC client.

Giai đoạn cấu hình:

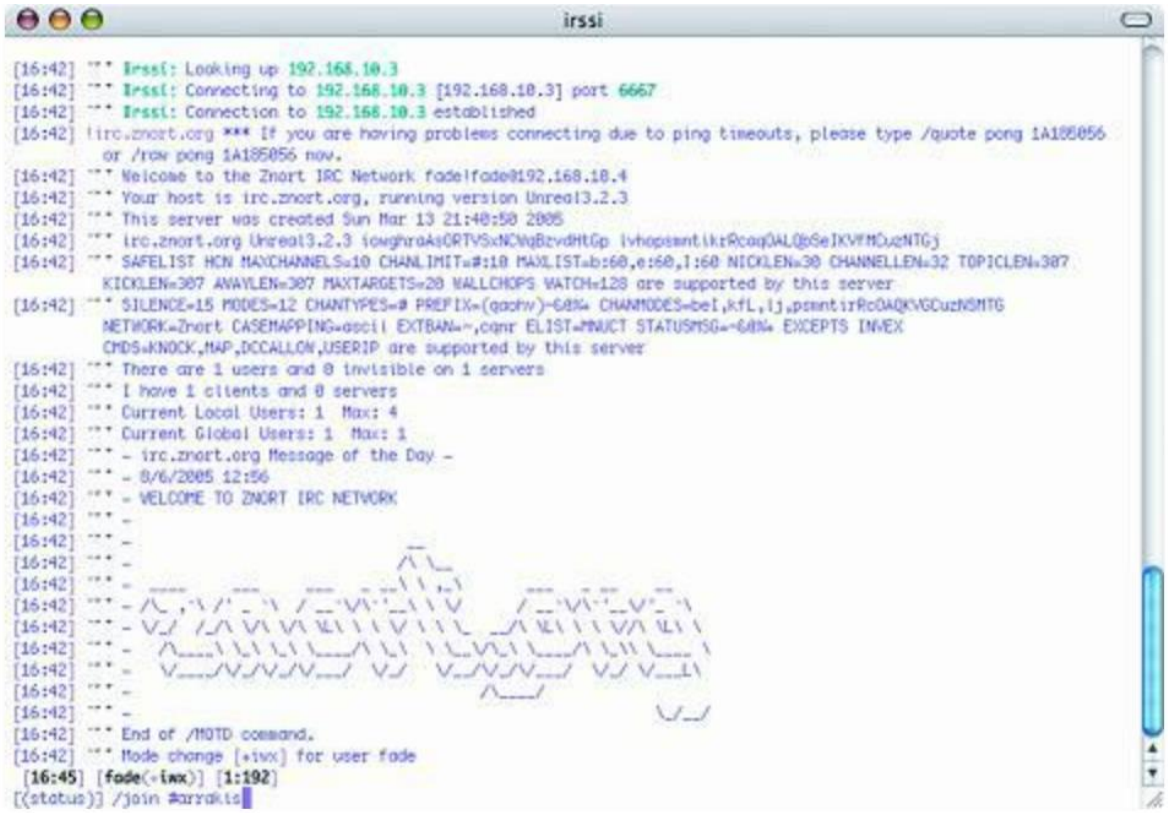
Bước đầu tiên là cấu hình bot thông qua giao diện đồ hoạ đơn giản của nó (xem hình bên dưới) . Thông tin được nhập vào bao gồm tên, số cổng IRC server, tên kênh, danh sách người sử dụng với mật khẩu master (chủ điều khiển), cuối cùng là tên file và thư mục cài đặt bot. Một số thành phần bổ sung cũng được kích hoạt như hỗ trợ sniffing và cơ chế trạng thái. Kết quả của giai đoạn này là file config. h, file nền tảng trong quá trình biên dịch bot được tạo.



Hình 1.5: Giao diện cấu hình thông tin của Agobot[10]

Giai đoạn điều khiển:

Sau khi bot được biên dịch, hai hệ thống còn lại sẽ tấn công theo kiểu "thủ công". Máy chủ (master) kết nối tới IRC server và liên kết với kênh dẫn để có thể ra lệnh điều khiển cho bot (xem hình dưới đây).



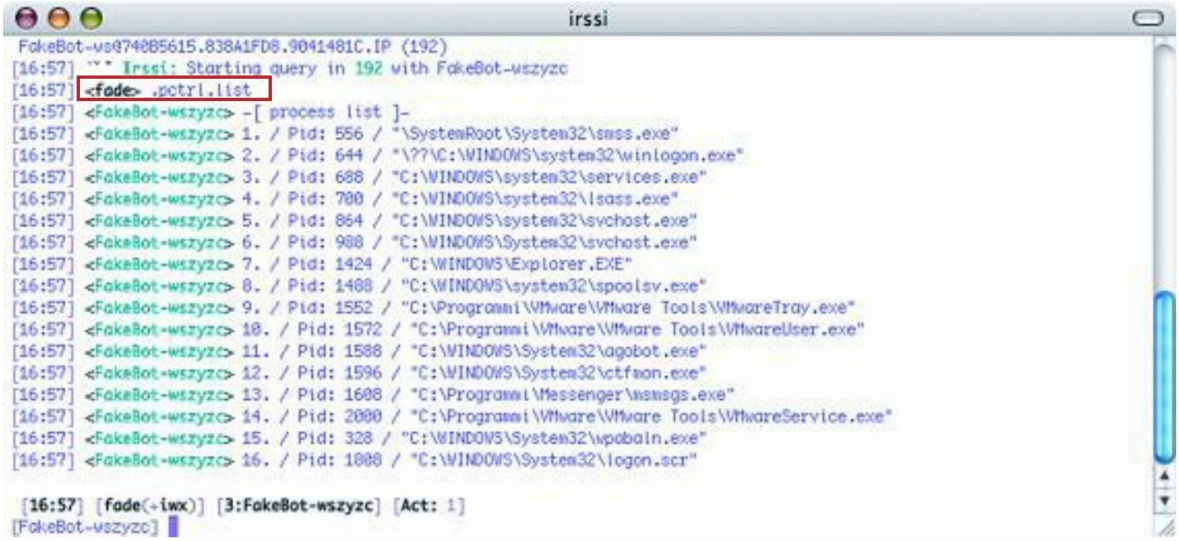
Hình 1.6: Botmaster kết nối IRC server và liên kết đến kênh dẫn để ra lệnh [10]

Muốn thu được quyền điều khiển qua các bot, một cơ chế thẩm định là cần thiết. Cơ chế này được tạo đơn giản bằng cách gửi một lệnh tới kênh (xem hình dưới đây).

Lệnh này là . *login* và cấu trúc như sau: . login "TênUser" "Password". Vì ở trên đã tạo tài khoản là Fade và mật khẩu là Dune nên lệnh sẽ là `.login FaDe dune`

Hình 1.7: Ta truy cập vào tài khoản botmaster đã tạo [10]

Sau đó, sử dụng bot đầu tiên được yêu cầu đưa ra danh sách tất cả chương trình đang chạy trên máy tính bị chiếm quyền kiểm soát bằng lệnh `. pctrl. list` (xem hình dưới đây):



Hình 1.8: Botmaster đưa ra lệnh đầu tiên cho bot con đầu tiên [10]

Sau đó, sử dụng bot thứ hai được yêu cầu đưa ra thông tin và khoá cdkey của các chương trình ứng dụng cài đặt trên máy bằng lệnh . *bot. sysinfo* và . *harvest. cdkeys* (xem hình dưới đây):

Hình 1.9: Botmaster đưa ra yêu cầu cho bot con thứ 2 của mình [10]

Ở ví dụ này chúng ta sử dụng những tính năng hết sức đơn giản. Còn thực tế, Agobot cung cấp một tập hợp rất phong phú các lệnh và chức năng. Một trong số chúng bạn có thể xem ở hình phía dưới đây [10].

Bảng 1.3: Danh sách lệnh mà Agobot thực thi được [10]

Lệnh	Mô tả
command.list	Danh sách tất cả các lệnh có thể sử dụng
bot.dns	Xử lý một địa chỉ IP hoặc hostname
bot.execute	Chạy một file .exe trên máy từ xa
bot.open	Mở một file trên máy từ xa
bot.command	Chạy một lệnh với system()
irc.server	Kết nối tới một IRC server
irc.join	Nhập thông tin của một kênh dẫn cụ thể
irc.privmsg	Gửi thư riêng cho một người dùng
http.execute	Download và thực thi file qua HTTP
ftp.execute	Download và thực thi file qua FTP
ddos.udpflood	Khởi động một chương trình UDP tràn
ddos.synflood	Khởi động một Syn tràn
ddos.phaticmp	Khởi động một PHATicmp tràn
redirect.http	Khởi động một HTTP proxy
redirect.socks	Khởi động một SOCKS4 proxy
pctrl.list	Đưa ra danh sách chương trình
pctrl.kill	Loại bỏ các chương trình

1.7 Giải pháp phòng chống DDoS

Luôn luôn kiểm tra lưu lượng mạng. Việc kiểm tra lưu lượng mạng là một công việc đơn giản nhưng khá quan trọng vì nó có thể giúp phát hiện lưu lượng mạng bất thường

Mở rộng băng thông. Tấn công DDoS là làm nghẽn lưu lượng mạng với việc gửi một lượng lớn gói tin từ đó không thể truy cập trang web được. Nhưng với việc mở rộng băng thông mạng thì việc làm nghẽn nó trở nên bất khả thi.

Luôn cập nhật thông tin các lỗ hỏng Zero day. Việc biết lỗ hỏng và các cách tấn công DDos mới nhất từ đó đưa ra kế hoạch sửa chữa lỗ hỏng bảo mật và phòng chống DDoS một cách hiệu quả nhất.

Cùng với đó sử dụng các máy chủ dự phòng cũng như là các phần mềm chống DDos mới nhất hiện nay để phòng ngờ trường hợp máy chủ bị sập.

CHƯƠNG 2: NGHIÊN CỨU CÁC THUẬT TOÁN MÁY HỌC TRONG PHÁT HIỆN TẤN CÔNG DDOS

2.1 Học có giám sát (Supervised Learining)

Đầu tiên theo nhóm về các thuật toán máy học trong phát hiện tấn công Ddos thì có những thuật toán sau: Decision tree, Random Forest, Navie Bayes, K-Nearest Neighbors(KNN), Logistic Regression, XGBoost. Đây đều là những thuật toán máy học phân loại trong học có giám sát vậy thì tại sao phải sử dụng những thuật toán này thì mới phát hiện được tấn công DDoS. Tại vì những hình thức như học không giám sát, học bán giám sát là không cho ta đầy đủ thông tin đầu vào từ đó ta không thể cho cho ra được kết quả chính xác, còn học củng cố thì khi máy phán đoán tính huống theo hoàn cảnh thì khi thực hiện hành động nào đó sẽ có thể tạo ra 1 vấn đề với hệ thống [11].

Học có giám sát (tiếng Anh là Supervised Learning) là một loại kỹ thuật trong lĩnh vực máy học có nhiệm vụ dự đoán được giá trị cũng như kết quả đầu ra 1 cách chính xác dựa trên cặp dữ liệu đã được cho trước hay còn gọi là dữ liệu training(input, come) hoặc (data, label) đã biết từ trước.

Biểu diễn toán về, học có giám sát là khi chúng ra có một tập hợp biến đầu vào là các cặp dữ liệu biết trước (xi, yi) $\in X \times Y$, x là đầu vào và y là đầu ra được gọi là tập training data (dữ liệu huấn luyện). Từ tập training data này, chúng ta cần tìm một hàm (function) là Y=f(x) . Mỗi phần tử từ tập X sang một phần tử tương ứng là tập Y [11]. Học có giám sát được chỉ nhỏ thành 2 dạng mô hình như sau:

Phân loại (Classification): Một bài toán được gọi là phân loại nếu những dữ liệu đầu vào được chia thành một số hữu hạn nhóm có thể xác định được. Ví dụ: Gmail xác định xem một email có phải là spam hay không; Phân loại các chuyên mục bài báo .

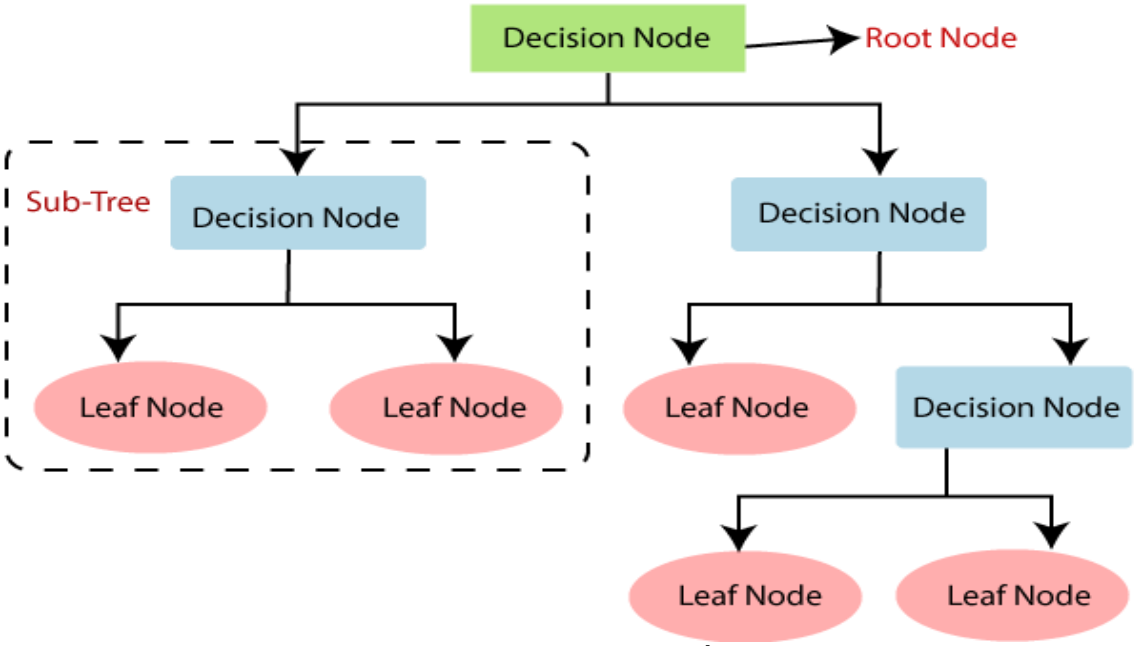
Hồi quy (Regression): Trường hợp này, label sẽ không chia thành các nhóm mà là một giá trị cụ thể. Ví dụ: xác định giới tính qua khuôn mặt trong ảnh, xác định xem thử đây có phải là con cá hay không thông qua các chi tiết mô tả sẵn như da, đuôi, với ngoại hình, vv. . .

2.2 Các thuật toán máy học phân loại trong học có giám sát

2.2.1 Decision Tree

Decision Tree (cây quyết định) là một trong những phương pháp học có giám sát có phi tham số được sử dụng trong phân loại và hồi quy đây là 1 thuật toán rất lâu đời với mục đích đưa ra được quyết định để xây dựng một kế hoạch nhằm đạt được mục tiêu mong muốn. Trong trường hợp phát hiện DDoS thì việc ứng dụng cây quyết định sẽ cho khi có signature DDoS trong hệ thống sẽ dễ giúp máy ra phát hiện DDoS [12].

Cây quyết định gồm có 3 phần chính 1 node gốc (root node), những node lá (leaf nodes) và các nhánh của nó (branches). Node gốc là điểm bắt đầu của cây quyết định và cả hai node gốc và node chứa câu hỏi hoặc tiêu chí để được trả lời. Nhánh biểu diễn các kết quả của kiểm tra trên nút [12].



Hình 2.1: Ví dụ cây quyết định

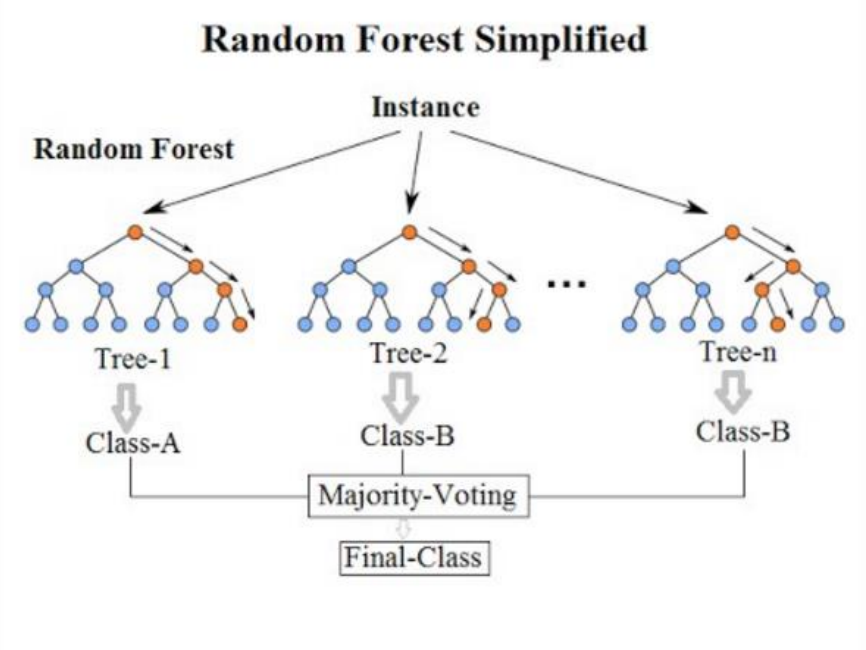
Ưu điểm: Đơn giản dễ hiểu, giúp ta có thể tính toán ra các trường hợp và lường trước các biến số, xử lý được nhiều loại dữ liệu vì là phi tham số xử lý được vấn đề đa lớp và bài toán phân loại nhiều lớp.

Nhược điểm: Khi bộ dữ liệu quá ngắn và ít thì sẽ không thể đưa ra được các quyết định chính xác, và nếu 1 dữ liệu xuất hiện quá nhiều trong bộ dữ liệu cây quyết định

sẽ ưu tiên chọn nó, điều này làm mất đi sự chính xác và việc thay đổi dữ liệu có thể khiến cây có cấu trúc không ổn định.

2.2.2 Random Forest

Thuật toán Random Forest (rừng ngẫu nhiên) là một thuật toán học có giám sát tổng hợp phân loại, hồi quy bao gồm một rừng tập hợp nhiều Decision Tree làm cho thuật toán này cho ra kết quả quyết định có tính chính xác cao hơn so với Decision Tree. Phân loại ở việc đầu ra của rừng ngẫu nhiên là tất cả các cây quyết định. Hồi quy chủ yếu ở dự đoán giá trị trung bình kết quả đầu ra của tất cả cây quyết định trong rừng ngẫu nhiên. Mỗi cây trong rừng cây ngẫu nhiên đều được phát triển ở mức độ lớn nhất có thể. Không sử dụng cắt tỉa [3].



Hình 2.2: Ví dụ về thuật toán Random Forest

Ở hình ví dụ trên ta có thể thấy thuật toán rừng ngẫu nhiễn có 3 cây quyết định đếu như nhau nhưng ở những trường hợp khác nhau mỗi cây cho ra một quyết định sau đó thì sẽ lọc ra và chọn quyết định có sự giống nhau nhất và từ đó đi ra quyết định cuối cùng cho rừng cây [15].

2.2.3 Navie Bayes

Thuật toán Navie Bayes là thuật toán máy học có giám sát dựa trên định lý Bayes để giải quyết các bài toán phân loại. Nó chủ yếu để sử dụng phân loại số lượng lớn dữ liệu . Thuật toán Navie Bayes này có nhược điểm là nó giả định tất cả thuộc tính đều là độc lập, vì vậy nó không thể tìm hiểu mối quan hệ của các thuộc tính. Nhưng tốc độ xử lý phân loại của nó thật rất ấn tượng hãy xem ví dụ sau đây [13].

Công thức định lý Bayes:

$$P(Ci\,|X) = \frac{P(X|Ci).P(Ci)}{P(X)}$$

- P(Ci|X): xác suất của mục tiêu Ci với điều kiện có đặc trưng X
- P(X|Ci): xác suất của đặc trưng X khi đã biết mục tiêu y
- P(Ci) gọi là xác suất của mục tiêu của đặc trưng Ci
- P(X) gọi là xác suất của mục tiêu của đặc trưng X

Bảng 2.1: Dữ liệu cho Ví dụ của Navie Bayes [13]

age	income	student	credit_rating	buy?
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Ví dụ: Với bảng dữ liệu ở trên, xác đình lớp của mẫu: X = (Age<=30, Income = Medium, Student = Yes, Credit_Rating = Fair) Sẽ rơi vào lớp nào?

Các lớp: – C1:buys_computer = ‘yes’ – C2:buys_computer = ‘no’

Tính giá trị xác suất trước cho mỗi phân lớp

P(C1) = 9/14

P(C2) = 5/14

Tính giá trị xác suất của mỗi giá trị thuộc tính đối với mỗi phân lớp

P(Age = Young|C1) = 2/9; P(Age = Young|C2) = 3/5

P(Income = Medium|C1) = 4/9; P(Income = Medium|C2) = 2/5

P(Student=Yes|C1) = 6/9; P(Student=Yes|C2) = 1/5

P(Credit_Rating=Fair|C1) = 6/9; P(Credit_Rating=Fair|C2) = 2/5

Tính toán xác suất có thể xảy ra của mẫu X đối với mỗi phân lớp:

Đối với phân lớp C1

P(X|C1)=P(Age=Young|C1). P(Income=Medium|C1). P(Student=Yes|C1). P(Cred it_Rating=Fair|C1) = (2/9). (4/9). (6/9). (6/9) = 0. 044

Đối với phân lớp C2 P(X|C2)=P(Age=Young|C2). P(Income=Medium|C2). P(Student=Yes|C2). P(Cred it_Rating=Fair|C2) = (3/5). (2/5). (1/5). (2/5) = 0. 019

Xác định phân lớp có thể nhất

Đối với phân lớp C1: P(C1). P(X|C1) = (9/14). (0, 044) = 0. 028

Đối với phân lớp C2: P(C2). P(X|C2) = (5/14). (0. 019) = 0. 007

Kết luận: X sẽ mua một máy tính (Thuộc phân lớp C1) [12]

2.2.4 K-nearest neighbors (KNN)

K-nearest neighbors là thuật toán máy học có giám sát. Thuật toán KNN cho rằng những dữ liệu tương tự nhau sẽ tồn tại gần nhau trong một không gian, từ đó công việc của chúng ta là sẽ tìm k điểm gần với dữ liệu cần kiểm tra nhất [19].

Cách thức hoạt động:

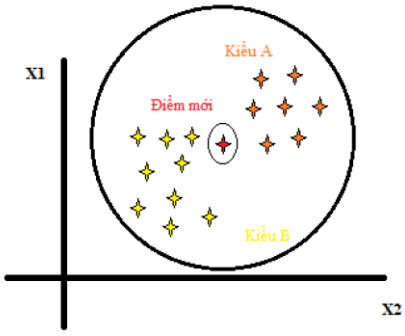
-Tính khoảng cách giữa d và tất cả các mẫu trong tập.

$$d = \sqrt{(X2 - X1)^2 + (Y2 - Y1)^2}$$

-Chọn k mẫu gần nhất với d trong tập.

-Gán d vào lớp có nhiều mẫu nhất trong số k mẫu láng giềng đó (hoặc d nhận giá trị trung bình của k mẫu)

Có một tập hợp các điểm dữ được chia theo các kiểu và thêm một điểm, vì vậy cần phân loại điểm dữ liệu mới thuộc kiểu A hay kiểu B



Hình 2.3: Điểm dữ liệu mới cần phân loại

Trước tiên, chọn số lượng các điểm gần với điểm mới với k=5. Sau đó tính khoảng cách euclid giữa các điểm của các kiểu.

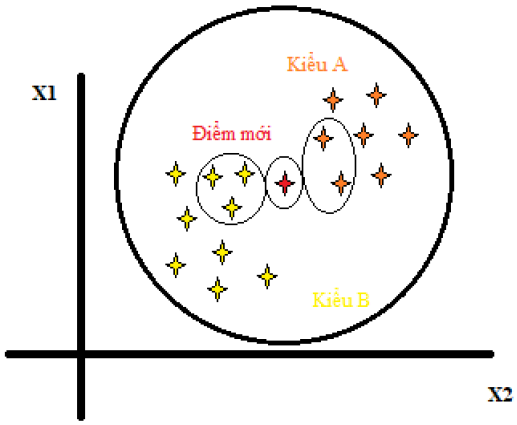
Công thức:

$$d = \sqrt{(X_1 - X_2)^2 + (Y_1 - Y_2)^2}$$

X là toạ độ của các điểm gần

Y là toạ độ của điểm dữ liệu mới

Tính toán xong sẽ có các điểm gần với điểm mới nhất với 3 điểm thuộc kiểu B và 2 điểm thuộc kiểu A gần với điểm mới nhất

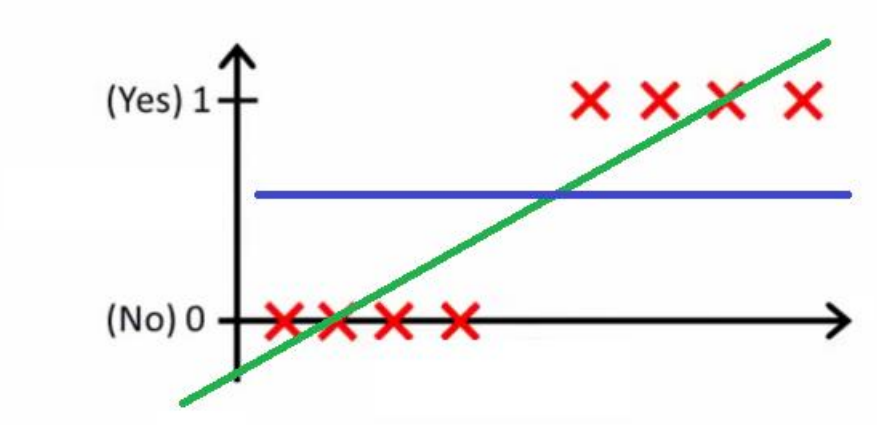


Hình 2.4: Điểm dữ liệu mới cần phân biệt thuộc kiểu B

Cuối cùng có thể nhận biết được rằng điểm dữ liệu mới gần với 3 điểm thuộc kiểu B nhất vì vậy điểm dữ liệu mới sẽ thuộc kiểu B.

2.2.5 Logistic Regression

Logistic Regression là một phương pháp phân tích thống kê được sử dụng để dự đoán giá trị dữ liệu dựa trên các quan sát trước đó của tập dữ liệu nhưng nó cũng có thể dùng cho bài toán phân loại [14].



Hình 2.5: Ví dụ về Logistic Regression

Logistic Regression bắt nguồn từ tên của nó từ hàm sigmoid, còn được gọi là hàm logistic. Hàm logistic là một đường cong hình chữ S trải dài từ 0 đến 1, đồng thời không bao giờ chính xác là 0 và cũng không bao giờ chính xác là 1.

Công thức Sigmoid Function:

$$S(x)=\frac{1}{1+e^{-x}}=\frac{e^{x}}{e^{x}+1}=1-S(-x)$$

2.2.6 XGBoost

XGBoost là một trong những kỹ thuật tăng cường gradient (gradient boost) có hiệu suất và tốc độ nâng cao trong các thuật toán học máy dựa trên cây quyết định (Decision Tree). XGBoost xây dựng một mở rộng bổ sung của hàm mục tiêu (objective function) bằng cách giảm thiểu hàm mất mát (loss function). Nó thường dựa vào cây quyết định (Decision Tree) làm bộ phân loại cơ sở [4].

Công thức:

$$L_{xgb} = \sum_{i=1}^{N} L\,(y_i, F(x_i)) + \sum_{i=1}^{N} \Omega\,(h_m)$$

$$\Omega(h) = \gamma T + \frac{1}{2}\lambda \|w\|^2$$

Trong đó:

-T là số lá của cây

-w là các điểm output của các lá.

-γ là giá trị kiểm soát mức tăng giảm tổn thất tối thiểu cần thiết để tách một nút bên trong. Giá trị γ càng cao thì cây càng đơn giản.

2.3 Các kỹ thuật sử dụng trong tiền xử lý dữ liệu

2.3.1 Kỹ thuật xử lý các giá trị rỗng trong bộ dữ liệu

Đối với một bộ dữ liệu lớn lên đến 10 ngàn hay trăm ngàn dữ liệu thì đội khi có các cột thuộc tính của vài dữ liệu bị rỗng thì ta sẽ thực hiện xử lí dữ liệu chúng ta có 4 loại xử lí giá trị rỗng sau đây:

Loại bỏ cột/thuộc tính có 1 hoặc nhiều giá trị rỗng:

Tên	Tuổi
Trí	21
Tài	21
NaN	25

Tuổi
21
21
25

Ưu điểm của loại kỹ thuật này là không có độ khó dễ dàng thực hiện nhưng mà nhược điểm của loại kỹ thuật này là dễ làm mất đi các cột dữ liệu trong bộ dữ liệu dễ biến 1 bộ dữ liệu thiếu sót thông tin và có chất lượng thấp khi sử dụng trong mô hình máy học .

Loại bỏ các cột, các mẫu có nhiều thuộc tính bị rỗng:

Tên	Tuổi
Trí	21
NaN	23
Tài	21
NaN	25

Tên	Tuổi
Trí	21
Tài	21

Cũng giống với kỹ thuật ở trên về mức độ rất đơn giản và dễ thực hiện được nhưng mà khác nhau với kỹ thuật ở trên là giữ lại các thuộc tính quan trọng của bộ dữ liệu chứ không xóa hoàn toàn mất đi như ở trên điều này sẽ khiến cho bộ dữ liệu được hoàn chỉnh. Nhược điểm dễ gây mất cân bằng cho bộ dữ liệu.

Thay thế giá trị rỗng bằng giá trị khác:

Tên	Tuổi
Trí	21
Quyền	NaN
Tài	21
Duy	NaN

Tên	Tuổi
Trí	21
Quyền	23
Tài	21
Duy	25

Thay vì ta xóa giá trị rỗng như ở trên thì ta có thể thay thế nó bằng 1 kiểu giá trị 0, 1, 2 … mà ta quy định sẵn hoặc trung bình cộng của các giá trị trong các cột thuộc tính. Ở loại giá trị này chỉ dùng để bổ sung cho đủ bộ dữ liệu dùng trong nghiên cứu chứ không giá trị thực tế vì là loại giá trị theo kiểu tự nghĩ ra hoặc tính toán hoặc ngẫu nhiên nên độ chính xác không cao hầu hết trong các trường hợp nhưng làm nghiên cứu mô hình máy học thì sẽ hoàn thiện và không bị thiếu bộ dữ liệu. Thậm chí còn thể thêm vào 1 cột tính năng cho biết đâu là giá trị vừa bị thay thế giá trị rỗng thêm vào.

2.3.2 Xử lý các biến giá trị phân loại

Mã hóa với giá trị số:

Mã hoá với giá trị số là dạng mã hoá tương tự như mã hoá thứ tự (là kỹ thuật mã hoá sẽ mã hoá mỗi giá trị khác nhau thành một giá trị nguyên duy nhất theo thứ tự) nhưng thay vì gán theo thứ tự có thể gán số tuỳ vào mục đích sử dụng. Ví dụ dưới thay thế các giá trị trong cột "Khu vực" thành các giá trị số tương ứng của các giao thức đó (Với Hà Nội "12", Tp. Hồ Chí Minh "5", Hoà Bình "5", Đà Lạt "6").

Khu Vực
Hà Nội
Tp. Hồ chí Minh
Hoà Bình
Tp. Hồ chí Minh
Đà Lạt
Đà Lạt

Khu Vực
12
5
10
5
6
6

Hình 2.6: Ví dụ về mã hóa với giá trị số

Mã hóa one-hot encoding:

One-hot encoding là một phương pháp **chuyển đổi dữ liệu**. Với one-hot, chúng chuyển đổi mỗi giá trị phân loại thành một cột phân loại mới và gán giá trị nhị phân là 1 hoặc 0 cho các cột đó. Giá trị sẽ là 1 nếu tương ứng với cột có thuộc tính gốc và ngược lại sẽ là 0 nếu không tương ứng. Nhược điểm của kỹ thuật này là tạo ra 1 số lượng lớn các cột làm tăng kích thước tệp dữ liệu và số lượng đầu vào của bài toán. [15]

Khu Vực
Hà Nội
Tp. Hồ chí Minh
Hoà Bình
Tp. Hồ chí Minh
Đà Lạt
Đà Lạt

Khu Vực - Hà Nội	Khu Vực - Tp.Hồ Chí Minh	Khu Vực - Hoà Bình	Khu Vực - Đà Lạt
1	0	0	0
0	1	0	0
0	0	1	0
0	1	0	0
0	0	0	1
0	0	0	1

Hình 2.7: Ví dụ về mã hóa one-hot encoding

2.3.3 Kỹ thuật Feature Selection

Phương pháp Chi-Squared

Chi-Square là kiểm định giả thuyết thống kê có giá trị để thực hiện khi thống kê kiểm định được phân phối chi bình phương theo giả thuyết rỗng, cụ thể là kiểm định chi-square của Pearson và các biến thể của chúng. Kiểm tra sự khác biệt của giá trị quan sát với giá trị kỳ vọng và cho ra kết quả kiểm tra được mối quan hệ của giữa 2 biến phân loại [17].

Công thức:

$$X_C^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Trong đó: c là độ tự do

Oi là giá trị quan sát được

Ei là độ kỳ vọng, giá trị kỳ vọng

Phương pháp ANOVA

ANOVA tên thật nghĩa là phân tích phương sai 1 phương pháp hay còn gọi là kiểm định ANOVA là một kỹ thuật thống kê tham số được sử dụng để so sánh các bộ dữ liệu. Các nhà phân tích sử dụng thử nghiệm ANOVA để xác định ảnh hưởng của các biến độc lập đối với biến phụ thuộc trong nghiên cứu hồi quy. Nó có thể phân biệt được giữa 2 hay nhiều means [20].

Công thức:

$$F = \frac{MSB}{MSW}$$

Trong đó: F là ANOVA Coefficient được sử dụng để so sánh các yếu tố độ lệnh gọi tắt là ANOVA – F.

MSB là tổng bình phương trung bình giữa các nhóm.

MSW là tổng bình phương trung bình trong các nhóm.

2.3.4 Kỹ thuật cân bằng dữ liệu Controlled Under-Sampling

Kỹ thuật này tạm dịch là kỹ thuật lấy mẫu có kiểm soát nằm trong thuật toán lựa chọn mẫu kĩ thuật này cho phép lấy mẫu trong đó người dùng chỉ định số lượng mẫu. Trong từ tập mẫu bạn đầu sẽ tạo ra tập mẫu mới giống vậy sao cho tập mẫu sau lớn hơn tập mẫu trước nhưng tập mẫu sau sẽ là con tập mẫu trước. Ở kỹ thuật này có 2 kỹ thuật khác nữa đó là RandomUnderSampler và Nearmiss.

RandomUnderSampler:

Đây là kỹ thuật cân bằng bộ dữ liệu 1 cách ngẫu nhiên, đơn giản và rất hiểu quả nó chọn tập hợp con dữ liệu từ bộ dữ liệu gốc của kĩ thuật Controlled Under-Sampling vì thuộc kĩ thuật này nên ta có thể chỉ định số lượng mẫu từ tập hợp có thể lấy dựa vào tham số sampling_strategy trong class RandomUnderSampler từ thư viện của sklearn thuộc imblearn. under_sampling

NearMiss:

Đây là kỹ thuật được tạo ra dựa trên thuật toán nearest neighbor. Kỹ thuật này sẽ lựa chọn mẫu trong các lớp nhiều mẫu nhất cho tới các lớp ít mẫu nhất từ đa số tới thiểu số. NearMiss có 3 dạng: NearMiss-1, NearMiss-2, NearMiss-3

NearMiss-1: Sẽ lấy mẫu từ lớp nhiều mẫu nhất có khoảng cách trung bình nhỏ đến 3 mẫu gần nhất của lớp ít mẫu nhất.

NearMiss-2: Sẽ lấy mẫu từ lớp nhiều mẫu nhất có khoảng cách trung bình nhỏ đến 3 mẫu xa nhất của lớp ít mẫu nhất.

NearMiss-3: Sẽ lấy mẫu nhất định cho lớp nhiều mẫu nhất với mẫu trong lớp ít mẫu nhất gần nhất .

2.3.5 Kỹ thuật Feature Scaling

Feature Scaling là một kỹ thuật để chuẩn hóa các tính năng độc lập có trong dữ liệu trong một phạm vi cố định. Nó được thực hiện trong quá trình tiền xử lý dữ liệu để xử lý các cường độ hoặc giá trị hoặc đơn vị rất khác nhau. Nếu Feature Scaling không được thực hiện, thì thuật toán máy học có xu hướng cân nhắc các giá trị lớn hơn, cao hơn và coi các giá trị nhỏ hơn là giá trị thấp hơn, bất kể đơn vị của giá trị là gì, do đó đây được xem là một kỹ thuật quan trọng trong giai đoạn tiền xử lý. Thông thường có 2 kiểu chia tỷ lệ (scaling) thông dụng là Min-Max normalization và standardization [21].

Min-Max Normalization

Normalization là một kỹ thuật chia tỷ lệ trong đó các giá trị được dịch chuyển và mở rộng lại để chúng kết thúc trong khoảng từ 0 đến 1 [21].

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Trong đó:

- X_{max} là giá trị lớn nhất của cột/tính năng
- X_{min} là giá trị nhỏ nhất của cột/tính năng

Khi giá trị của X là giá trị tối thiểu trong cột, số sẽ là 0, và do đó X là 0. Mặt khác, khi giá trị của X là giá trị tối đa trong cột, số bằng với mẫu số và do đó giá trị của X là 1 . Nếu giá trị của X nằm giữa giá trị tối thiểu và tối đa, thì giá trị của X là từ 0 đến 1

Standardization

Standardization là một kỹ thuật chia tỷ lệ khác, trong đó các giá trị được tập trung xung quanh trung bình với độ lệch chuẩn đơn vị. Điều này có nghĩa là trung bình của thuộc tính trở thành số không và phân phối kết quả có độ lệch chuẩn đơn vị [21].

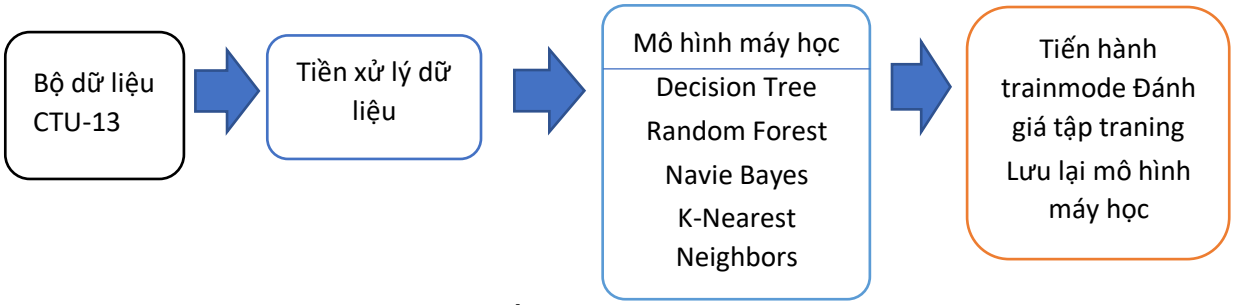
$$X' = \frac{X - \mu}{\sigma}$$

Trong đó:

- μ là giá trị trung bình của các giá trị trong cột/ tính năng.
- σ là độ lệch chuẩn của các giá trị trong cột/ tính năng

CHƯƠNG 3: XÂY DỰNG MÔ HÌNH MÁY HỌC VÀ THỰC NGHIỆM

3.1 Xây dựng mô hình máy học



Hình 3.1: Sơ đồ xây dựng các mô hình máy học

Có rất nhiều bộ dữ liệu hỗ trợ khác nhau trên hiện nay nhưng thực sự để chạy được mô hình máy học như hiện nay thì sklearn, bộ thư viện theo nhóm tìm hiểu là dễ hiểu nhất cho thuật toán cây quyết định, rừng cây ngẫu nhiên, … . Đây là thứ giúp cho thực hiện thử nghiệm trở nên 1 cách dễ dàng hơn trong việc xây dựng mô hình máy học trong các mục đích sử dụng, có thể nhiều mục đích như AI hay là thử nghiệm phòng chống Ddos. Bộ dữ liệu được sử dụng qua quá trình tiền xử lý dữ liệu rồi sau đó mới thực nghiệm dùng bộ dữ liệu chạy thử thuật toán tranning thử trên google colab hoặc 1 nền tảng khác cho phép tranning model máy học. Chúng tôi sẽ sử dụng 4 thuật toán để thử nghiệm mô hình máy học lần này. Sau đây chúng tôi sẽ nói về bộ dữ liệu CTU-13 được sử dụng trong mô hình máy học lần này [22].

3.2 Bộ dữ liệu CTU-13

CTU-13 là một tập dữ liệu về lưu lượng botnet đã được thu thập tại Đại học CTU, Cộng hòa Séc, vào năm 2011. Mục tiêu của tập dữ liệu là thu thập được một lượng lớn lưu lượng botnet thực kết hợp với lưu lượng thông thường và lưu lượng nền. Bộ dữ liệu CTU-13 bao gồm 13 kịch bản của các mẫu botnet khác nhau. Trong mỗi tình huống, chúng tôi đã thực thi một phần mềm độc hại cụ thể, phần mềm này sử dụng một số giao thức và thực hiện các hành động khác nhau. Bảng dưới đây cho thấy các đặc điểm của các kịch bản mạng botnet [22].

Bảng 3.1: Table 2 - Characteristic of the botnet scenarios [22]

Table 2 – Characteristics of the botnet scenarios. (CF: ClickFraud, PS: Port Scan, FF: FastFlux, US: Compiled and controlled by us.)

Id	IRC	SPAM	CF	PS	DDoS	FF	P2P	US	HTTP	Note
1	√	√	√							
2	√	√	√							
3	√			√				√		
4	√				√			√		UDP and ICMP DDoS.
5		√		√					√	Scan web proxies.
6				√						Proprietary C&C. RDP.
7									√	Chinese hosts.
8				√						Proprietary C&C. Net-BIOS, STUN.
9	√	√	√	√						
10	√				√			√		UDP DDoS.
11	√				√			√		ICMP DDoS.
12							√			Synchronization.
13		√		√					√	Captcha. Web mail.

Mỗi kịch bản được ghi lại trong một tệp pcap chứa tất cả các gói của ba loại lưu lượng. Các tệp pcap này đã được xử lý để thu được loại thông tin khác, chẳng hạn như NetFlows, WebLogs, v. v. Phân tích đầu tiên của bộ dữ liệu CTU-13, đã được mô tả và xuất bản trong bài báo "So sánh thực nghiệm các phương pháp phát hiện botnet" (xem Trích dẫn dưới đây) đã sử dụng NetFlows một chiều để biểu diễn lưu lượng và gán nhãn. Không nên sử dụng các NetFlows một chiều này vì chúng hoạt động tốt hơn so với phân tích thứ hai của chúng tôi về tập dữ liệu, sử dụng NetFlows hai chiều. NetFlows hai chiều có một số ưu điểm so với các NetFlows định hướng. Đầu tiên, họ giải quyết vấn đề phân biệt giữa máy khách và máy chủ, nhãn chi tiết hơn nhiều. Phân tích thứ hai của tập dữ liệu với NetFlows hai chiều là phân tích được xuất bản ở đây. Mối quan hệ giữa thời gian của các kịch bản số lượng gói tin, số lượng NetFlows và kích thước của các tệp pcap đều được hiển thị rõ ràng ở bảng bên dưới [22].

Bảng 3.2: Lưu lượng dữ liệu được capture lại của mỗi kịch bản [22]

Id	Duration(hrs)	# Packets	#NetFlows	Size	Bot	#Bots
1	6.15	71,971,482	2,824,637	52GB	Neris	1
2	4.21	71,851,300	1,808,123	60GB	Neris	1
3	66.85	167,730,395	4,710,639	121GB	Rbot	1
4	4.21	62,089,135	1,121,077	53GB	Rbot	1
5	11.63	4,481,167	129,833	37.6GB	Virut	1
6	2.18	38,764,357	558,920	30GB	Menti	1
7	0.38	7,467,139	114,078	5.8GB	Sogou	1
8	19.5	155,207,799	2,954,231	123GB	Murlo	1
9	5.18	115,415,321	2,753,885	94GB	Neris	10
10	4.75	90,389,782	1,309,792	73GB	Rbot	10
11	0.26	6,337,202	107,252	5.2GB	Rbot	3
12	1.21	13,212,268	325,472	8.3GB	NSIS.ay	3
13	16.36	50,888,256	1,925,150	34GB	Virut	1

Bảng 3.3: Phân bố các label trong NetFlows của từng kịch bản [22]

Scen.	Total Flows	Botnet Flows	Normal Flows	C&C Flows	Background Flows
1	2,824,636	39,933(1.41%)	30,387(1.07%)	1,026(0.03%)	2,753,290(97.47%)
2	1,808,122	18,839(1.04%)	9,120(0.5%)	2,102(0.11%)	1,778,061(98.33%)
3	4,710,638	26,759(0.56%)	116,887(2.48%)	63(0.001%)	4,566,929(96.94%)
4	1,121,076	1,719(0.15%)	25,268(2.25%)	49(0.004%)	1,094,040(97.58%)
5	129,832	695(0.53%)	4,679(3.6%)	206(1.15%)	124,252(95.7%)
6	558,919	4,431(0.79%)	7,494(1.34%)	199(0.03%)	546,795(97.83%)
7	114,077	37(0.03%)	1,677(1.47%)	26(0.02%)	112,337(98.47%)
8	2,954,230	5,052(0.17%)	72,822(2.46%)	1,074(2.4%)	2,875,282(97.32%)
9	2,753,884	179,880(6.5%)	43,340(1.57%)	5,099(0.18%)	2,525,565(91.7%)
10	1,309,791	106,315(8.11%)	15,847(1.2%)	37(0.002%)	1,187,592(90.67%)
11	107,251	8,161(7.6%)	2,718(2.53%)	3(0.002%)	96,369(89.85%)
12	325,471	2,143(0.65%)	7,628(2.34%)	25(0.007%)	315,675(96.99%)
13	1,925,149	38,791(2.01%)	31,939(1.65%)	1,202(0.06%)	1,853,217(96.26%)

Từ bảng trên ta thấy rõ sự phân bổ chệch lệch rất lớn giữa các luồng Background so với các luồng dữ liệu khác . Vấn đề này hà có gây ra sư mất ổn định cân bằng dữ liệu trên bộ dữ liệu CTU-13 nên khi đưa vào các mô hình máy học phải xử lý thật ổn. Sau đây bảng dưới đây là chi tiết các cột dữ liệu của bộ dữ liệu.

Bảng 3.4: Mô tả chi tiết các cột dữ liệu

Tên cột	Mô tả chi tiết
StartTime	Thời gian bắt đầu của một kịch bản tấn công, thời gian này tùy theo từng bản ghi nên sẽ khác nhau
Dur	Thời gian tấn công nhưng được ghi dưới dạng giây
Proto	Gồm nhiều giao thức được sử dụng trong tấn công
SrcAddr	Địa chỉ nguồn
Sport	Source Port – nguồn port
Dir	Hướng đi của traffic
Dst	Địa chỉ đích
Dport	Port đích
State	Trạng thái của các cổng theo giao thức có tổng 231 giá trị duy nhất
sTos	Source type of source trường giá trị này có các giá trị: 0, 1, 2, 3, 192, NaN
dTos	Destination type of source trường giá trị này có các giá trị: 0, 1, 2, 3, NaN
TotPkts	Tổng số gói tin có giá trị từ 1 đến 2686731
TotBytes	Tổng số bytes có giá trị từ 60 đến 2689640464
SrcBytes	Tổng số bytes bắt đầu từ nguồn có giá trị từ 0 đến 2635366235
Label	Đại diện cho 3 thuộc tính Background, Botnet và Normal

Sau đây sẽ là giải thích quá trình tạo lập thực nghiệm trên google colab của nhóm ở trên là giải thích cho các cột dữ liệu được sử dụng trong mô hình máy học của nhóm.

3.3 Giải thích thực nghiệm triển khai mô hình máy học

3.3.1 Tiền xử lý dữ liệu

Tại sao phải tiền xử lý dữ liệu ? Vì một bộ dữ liệu thô ban đầu sẽ có thể có nhiều dữ liệu sạn ta cần tiền xử lý dữ liệu để dữ liệu sạch hơn và phù hợp với mô hình máy học của chúng ta. Các bước dưới đây sẽ chuẩn hóa lại bộ dữ liệu giúp bộ dữ liệu đầy đủ thông tin số, không bị rỗng, không có bị nhiễu bộ dữ liệu.

3.3.1.1 Xóa các cột dữ liệu không cần thiết

Chúng em sẽ loại bỏ các cột / tính năng không cần thiết như ‘StartTime’, ‘SrcAddr’, ‘Sport’, ‘DstAddr’, ‘Dport’, ‘State’. Loại bỏ ‘StartTime’ vì đây chỉ là giá trị ngày giờ vì nó không có giá trị cho các mô hình machine learning. Bên cạnh đó việc loại bỏ 5 cột / tính năng ‘SrcAddr’, ‘Sport’, ‘DstAddr’, ‘Dport’, ‘State’ cũng giúp giảm kích thước dữ liệu và giảm thời gian cần đạo tạo mô hình machine learning.

3.3.1.2 Giải quyết các cột có giá trị bị rỗng

Sau khi loại bỏ các tính năng không cần thiết thì chúng em xử lý tiếp các giá trị rỗng. Các giá trị rỗng tồn tại trong 2 cột / tính năng là ‘sTos’ và ‘dTos’. Cách đơn giản để giải quyết vấn đề này là xoá đi 2 cột / tính năng ‘sTos’ và ‘dTos’. Nhưng điều này có thể làm giảm hiệu suất của mô hình máy học vì ngoại trừ các giá trị rỗng ra nó còn chứa những giá trị khác. Vì thế chúng em chọn giải pháp thay thế các giá trị rỗng bằng giá trị khác, cụ thể chúng em sẽ thay thế các giá trị rỗng bằng một số nguyên là 9 để tránh bị trùng với các giá trị khác của ‘sTos’ và ‘dTos’.

3.3.1.3 Giải quyết các dữ liệu nhãn thô (raw label)

Cột / tính năng ‘Label’ (cột lớp mục tiêu) trong bộ dữ liệu CTU-13 chứa các nhãn được gán dưới dạng dữ liệu thô mô tả đặc tính của từng luồng tương ứng. Để các mô hình máy học có thể sử dụng được cột / tính năng này thì chúng ta phải chuyển đổi các giá trị thô trên thành các giá trị số nguyên. Các giá trị này đại diện cho các lớp mà mô hình máy học sẽ phân loại ra dựa trên các cột còn lại. Chúng em sẽ gán 3 giá trị là 0, 1, 2 cho các nhãn thô tương ứng. Trong đó, các giá trị thuộc về lưu lượng Background sẽ là 0, lưu lượng Botnet sẽ là 1 và lưu lượng Normal sẽ là 2.

Label
flow=Background-Established-cmpgw-CVUT
flow=Background-Established-cmpgw-CVUT
flow=Background-TCP-Attempt
flow=Background-TCP-Attempt
flow=Background-TCP-Attempt
flow=Background-TCP-Attempt
flow=Background-Established-cmpgw-CVUT
flow=Background-Established-cmpgw-CVUT
flow=Background-Established-cmpgw-CVUT
flow=Background-TCP-Established

Hình 3.2: Cột tính năng label ở dữ liệu thô

Lưu ý: nhãn của các luồng do phần mềm độc hại tạo ra sẽ bắt đầu bằng "From-Botnet". Các nhãn "To-Botnet" là các luồng được gửi đến botnet bởi các máy tính không xác định, vì vậy chúng không được xem là lưu lượng độc hại.

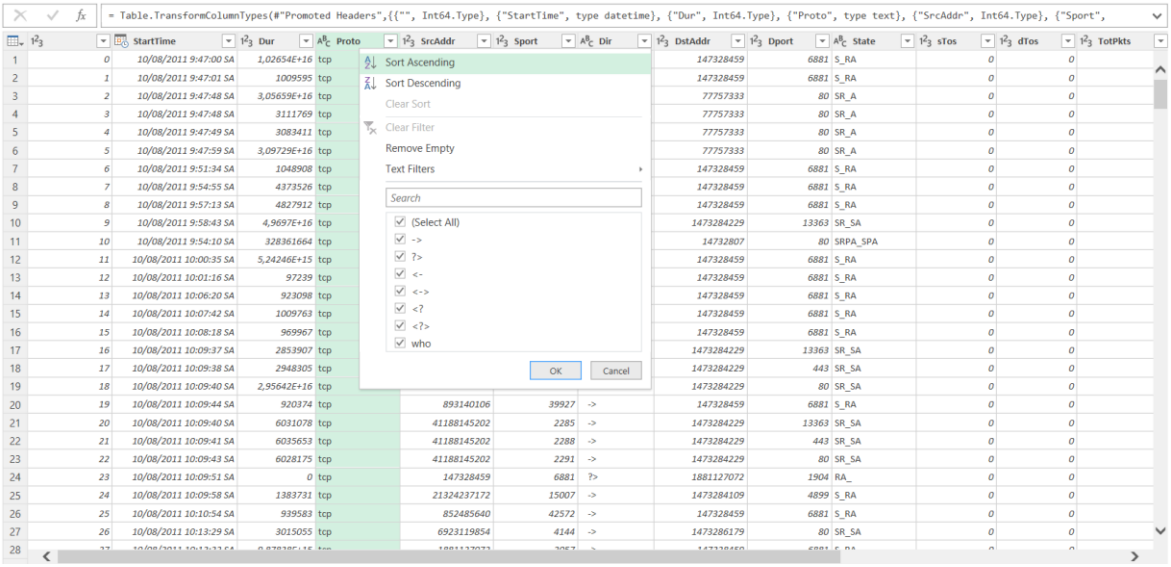
3.3.1.4 Giải quyết các biến giá trị phân loại

Bước tiếp theo của quá trình tiền xử lý là xử lý các biến/ giá trị phân loại. Các biến/ giá trị phân loại này tồn tại trong 4 cột / tính năng bao gồm 'Dir', 'Proto', 'sTos' và 'dTos'. Chúng em sẽ sử dụng 2 kỹ thuật đã trình bày ở trên để thực hiện bước này. Lưu ý: việc chúng em sử dụng 2 kỹ thuật là do chúng em sẽ chia ra thành 2 trường hợp để chuẩn bị cho các kịch bản sẽ trình bày trong phần sau.

Trường hợp 1: Sử dụng kỹ thuật mã hoá với giá trị số

Hai cột / tính năng 'sTos' và 'dTos' cơ bản nó chứa các giá trị số (xem như các biến phân loại) nên sẽ bỏ qua. Với cột / tính năng 'Dir' có chứa tổng cộng 7 biến giá trị phân loại bao gồm '->', '?>', '<->', '<?>', 'who', '<', '<?'. Trong đó giá trị '<->' chiếm khoảng 71. 77% và giá trị '->' chiếm khoảng 26. 89% so với các giá trị còn lại. Do đó, chúng em sẽ mã hoá riêng cho giá trị '<->' = 1 và giá trị '->' = 2, còn tất cả các giá trị còn lại sẽ được mã hoá = 0 do tỉ lệ phần trăm thấp so với tổng số 2 giá trị kia. Tương tự với cột / tính năng 'Proto' có chứa tổng cộng 15 biến giá trị phân loại bao gồm 'tcp', 'udp', 'rtp', 'pim', 'icmp', 'arp', 'ipx/spx', 'rtcp', 'igmp', 'ipv6-icmp', 'ipv6', 'udt', 'esp', 'unas', 'rarp'. Trong đó giá trị 'udp' chiếm khoảng 75. 20%, giá trị 'tcp' chiếm khoảng 21. 92% và giá trị 'icmp' chiếm khoảng 2. 48% so

với các giá trị còn lại. Lúc này chúng em cũng thực hiện giống với cột / tính năng ‘Dir’. Đó là mã hóa riêng cho các giá trị ‘udp’ = 17, giá trị ‘tcp’ = 6, giá trị’icmp’= 1 và tất cả các giá trị còn lại sẽ được mã hoá = 0.



Hình 3.3: Giá trị nằm ở cột Dir và Proto ở dữ liệu thô

Trường hợp 2: Sử dụng kỹ thuật mã hoá One-hot Encoding

Nhóm chúng em thực hiện kỹ thuật này theo cách như sau. Cụ thể là chúng em vẫn sử dụng mã hoá One-hot Encoding cho cột ‘Proto’ thành 4 cột ‘Proto_udp’, ‘Proto_tcp’, ‘Proto_icmp’ và ‘Proto_others’, cột ‘Dir’ thành 3 cột ‘Dir_ <->’, ‘Dir_ ->’ và ‘Dir_others’. Đều dựa vào bài báo [3] mà nhóm tham khảo được để thực hiện lại.

Vấn đề mã hoá như vậy là do sự chệnh lệch các giá trị trong 2 cột trên nên những cột có giá trị với số lượng lớn hơn các giá trị khác thì sẽ được mã hoá One-hot thành 1 cột riêng, còn những giá trị với số lượng nhỏ còn lại sẽ được gom lại để mã hoá thành cột “others”. Còn 2 cột/ tính năng ‘sTos’ và ‘dTos’ thì với ‘sTos’ mã hoá thành 3 cột ‘sTos_0. 0’, ‘sTos_9. 0’, ‘sTos_others’, tương tự với ‘dTos’ sẽ được mã hoá thành 3 cột ‘dTos_0. 0’, ‘dTos_9. 0’, ‘dTos_others’. Trong đó, 2 cột ‘sTos_9. 0’ và ‘dTos_9. 0’ là 2 cột được mã hoá từ giá trị rỗng đã được thay thế bằng số nguyên là 9 trong bước xử lý các giá trị rỗng.

3.3.1.5 Thực hiện kỹ thuật Feature Selection

Trường hợp 1: Phương pháp Chi-Squared

Bộ dữ liệu CTU-13 sau khi được mã hoá bằng One-hot Encoding sẽ bao gồm 17 cột /tính năng sau: ‘Dur’, ‘TotPkts’, ‘TotBytes’, ‘SrcBytes’, ‘Dir_ <->’, ‘Dir_ - >’, ‘Dir_others’, ‘Proto_udp’, ‘Proto_tcp’, ‘Proto_icmp’, ‘Proto_others’, ‘sTos_0. 0’, ‘sTos_9. 0’, ‘sTos_others’, ‘dTos_0. 0’, ‘dTos_9. 0’, ‘dTos_others’.

Sau khi cân bằng tập dữ liệu thì chúng em sẽ sử dụng lớp SelectKBest với tham số chi2 đại diện cho phương pháp Chi-Squared và chọn ra 10 cột / tính năng tốt nhất với tham số k=10. Cuối cùng, 10 cột tính năng đã được chọn là ‘Dur’, ‘TotPkts’, ‘TotBytes’, ‘SrcBytes’, ‘Dir_->’, ‘Dir_<->’, ‘Proto_icmp’, ‘Proto_tcp’, ‘Proto_udp’, ‘dTos_9. 0’.

Trường hợp 2: Phương pháp ANOVA

Tương tự với trường hợp phương pháp Chi-Squared, chúng em cũng sẽ dùng lại lớp SelectKBest với tham số f_classif đại diện cho phương pháp ANOVA và chọn ra 10 cột / tính năng tốt nhất với tham số k=10. Cuối cùng, 10 cột tính năng đã được chọn là ‘Dur’, ‘Dir_others’, ‘Dir_->’, ‘Dir_<->’, ‘Proto_icmp’, ‘Proto_tcp’, ‘Proto_udp’, ‘sTos_0. 0’, ‘dTos_0. 0’, ‘dTos_9. 0’.

3.3.1.6 Cân bằng lại bộ dữ liệu

Ta sẽ tiến hành cân bằng lại bộ dữ liệu vì lớp Background có 1 số lượng mẫu cực lớn nên phải cân bằng lại so với những lớp mẫu khác và nhóm em sẽ xử dụng kĩ thuật RandomUnderSampler và kĩ thuật NearMiss-1.

Trường hợp 1: Sử dụng kỹ thuật RandomUnderSampler

Ta sẽ sử dụng RandomUnderSampler từ thư viện imblearn. under_sampling thư viện này cho phép ta lấy mẫu ngẫu nhiên ở các lớp đa số hay thiểu số theo ý của mình. Chúng tôi sẽ lấy mẫu sao cho vẫn giữ nguyên số lượng mẫu thiểu số ở lớp Botnet là 444699 mẫu và lớp Normal là 356433 mẫu, còn với lớp có số lượng mẫu nhiều như lớp Background thì sẽ giảm đi từ 19175551 xuống 801132 bằng với tổng lượng mẫu của lớp Botnet và lớp Normal cộng lại.

Trường hợp 2: Sử dụng kĩ thuật NearMiss-1

Kỹ thuật này dựa vào thuật toán nearest neighbors. Nhưng với bài này thì sẽ sử dụng lấy lượng mẫu tượng tự như với kỹ thuật RandomUnderSampler

3.3.1.7 Tiến hành chia tỉ lệ cho bộ dữ liệu

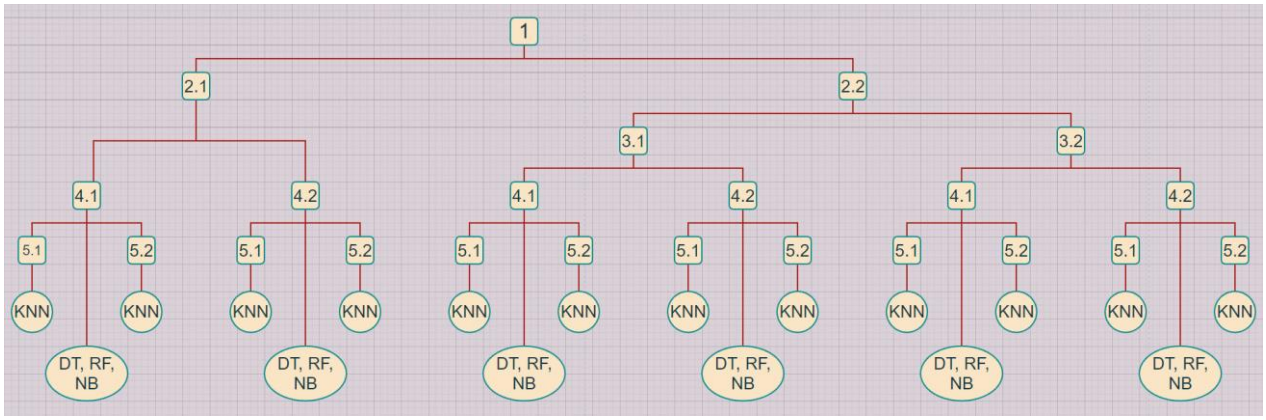
Trường hợp 1: Sử dụng kỹ thuật Normalization

Ở kỹ thuật này chúng tôi sẽ sử dụng lớp MinMaxScaler từ thư viện của sklearn. preprocessing . Nó sẽ chia tỷ lệ bằng cách tìm giá trị nhỏ nhất và lớn nhất của bộ dữ liệu sau đó dùng chúng để chia tỉ lệ cho từng cột dữ liệu trong bộ dữ liệu

Trường hợp 2: Sử dụng kỹ thuật Standardization

Giống với kỹ thuật ở trên, kỹ thuật này sẽ sử dụng lớp StandardScaler thông qua thư viện sklearn. preprocessing. Làm vậy sẽ giúp thay đổi hình dạng phân bổ của dữ liệu.

3.3.2 Mô hình kịch bản sử dụng để thực nghiệm



Hình 3.4: Mô hình kịch bản thực nghiệm[18]

(1): đọc dữ liệu sau đó, xoá các cột/ tính năng không cần thiết, gán lại nhãn và xử lý các giá trị rỗng

(2. 1): Mã hoá các biến/ giá trị phân loại với giá trị số

(2. 2): Mã hoá các biến/ giá trị phân loại với One-hot Encoding

(3. 1): Lựa chọn cột/ tính năng với Chi-Squared.

(3. 2): Lựa chọn cột/ tính năng với ANOVA.

(4. 1): Cân bằng bộ dữ liệu với RandomUnderSampler

(4. 2): Cân bằng bộ dữ liệu với NearMiss-1.

(5. 1): Chia tỉ lệ của các cột với Normalization.

(5. 2): Chia tỉ lệ của các cột/ tính năng với Standardization

DT là Decision Tree với Gini Impurity, RF là Random Forest với với Gini Impurity, NB là Naive Bayes với dùng phân phối Gaussian, KNN là K-Nearest Neighbors với K= 5 là mặc định.

Tất cả thực nghiệm đều được mô tả chi tiết ở mục 3.3.2, các thuật toán tiền xử lý dữ liệu đều được xử lý như nhau theo các kịch bản.

3.4 Tiến hành đánh giá kết quả thực nghiệm

Để tiến hành đánh giá được kết quả nhóm em đã thêm 1 thuật toán đánh giá vào với công thức như sau:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Trong đó:

True Positive (TP) là trường hợp dự đoán đúng nhãn là True và nhãn thực tế là True

True Negative (TN) là trường hợp dự đoán đúng nhãn là False và nhãn thực tế là False

False Positive (FP) là trường hợp dự đoán sai nhãn là True và nhãn thực tế là False.

False Negative (FN) là trường hợp dự đoán sai nhãn là False và nhãn thực tế là True.

Sau đây tiến hành đánh giá kết quả kịch bản thực nghiệm

3.4.1 Kết quả trường hợp kịch bản sử dụng mã hóa với giá trị số

Trường hợp mã hóa với giá trị số sẽ cho ra các kết quả như sau:

Bảng 3.5: Trường hợp độ chính xác với mã hóa với giá trị số

KỊCH BẢN				Độ chính xác	Thời gian chạy (giây)	% độ chính xác / giây
2.1	4.1	None	DT	90.71%	142	0.006388239
			RF	92.04%	315	0.00648169
			NB	27.41%	136	0.001930282
		5.1	KNN	87.88%	500	0.006188732
		5.2	KNN	91.38%	320	0.006435211
	4.2	None	DT	94.14%	428	0.006629859
			RF	94.55%	547	0.006658451
			NB	57.60%	472	0.004056338
		5.1	KNN	93.29%	720	0.006569718
		5.2	KNN	93.83%	740	0.006607746

Đánh giá: Sau khi sử dụng 4 thuật toán để chạy thì thuật toán Random Forest, DecisionTree, K-Nearest Neighbors là những thuật toán có độ chính xác cao. Trong

đó Random Forest có độ chính xác cao nhất với kịch bản (4.2) và độ chích sác thấp nhất là Navie Bayes với độ chính xác là 27.41%

3.4.2 Kết quả trường hợp kịch bản sử dụng mã hóa one-hot coding

Khi sử dụng kỹ thuật Chi-Squared để lựa chọn tính năng

Bảng 3.6: Trường hợp kỹ thuật lựa chọn tính năng Chi-Squared

	KỊCH BẢN			Độ chính xác	Thời gian chạy (giây)	% độ chính xác / giây
2.2 / 3.1	4.1	None	DT	90.72%	231	0.003927273
			RF	92.04%	519	0.00177341
			NB	27.41%	216	0.001268981
		5.1	KNN	87.87%	920	0.000955109
		5.2	KNN	91.14%	860	0.001059767
	4.2	None	DT	94.15%	492	0.001913618
			RF	94.59%	746	0.001267962
			NB	57.61%	563	0.001023268
		5.1	KNN	93.28%	920	0.001013913
		5.2	KNN	93.95%	923	0.001017876

Đánh giá: Sau khi sử dụng 4 thuật toán trên thì chúng tôi thấy mô hình có độ chính xác cao nhất là Random Forest là 94.59%. Decision Tree cũng không kém cạnh với tỉ lệ 94.15% và mô hình tỉ lệ thấp nhất là Navie Bayes là 27.41% điểm chung là những mô hình tốt nhất hiệu suất cao đều sử dụng (4.2) có kỹ thuật cân bằng dữ liệu Near-Miss

Khi sử dụng kỹ thuật ANOVA để lựa chọn tính năng

Bảng 3.7: Trường hợp sử dụng kỹ thuật lựa chọn tính năng ANOVA

	KỊCH BẢN			Độ chính xác	Thời gian chạy (giây)	% độ chính xác / giây
2.2 / 3.2	4.1	None	DT	69.73%	206	0.003384951
			RF	69.87%	583	0.001198456
			NB	42.43%	214	0.00198271
		5.1	KNN	69.87%	1220	0.000572705
		5.2	KNN	69.89%	1580	0.000442342
	4.2	None	DT	77.65%	3036	0.000255764
			RF	77.66%	3212	0.000241781
			NB	65.82%	2280	0.000288684
		5.1	KNN	76.06%	3020	0.000251854
		5.2	KNN	76.43%	3320	0.000230211

Đánh giá: Sau khi sử dụng 4 thuật toán trên thì chúng tôi thấy tổng quan chất lượng đều bị thấp đi rất nhiều có thể do quá trình xóa đi một số cột làm cho việc sử dụng ANOVA khiến mô hình có độ chính xác bị giảm đi. Mô hình tốt nhất vẫn là Random

Forest với 77.66% theo sau là Decision Tree khi sử dụng (4.2) và thấp nhất vẫn là Navie Bayes với 42.43% khi sử dụng (4.1).

KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

Sau quá trình thực hiện đề tài thì nhóm chúng em sẽ đưa ra kết luận Mặc dù tấn công Ddos/Botnet là một hình thức tấn công theo nhiều hướng khác nhau, dễ có và khó có. Để tìm ra cách ngăn chặn cũng như phát hiện Ddos/Botnet thì ta cần có kiến thức nền tảng về nó sau khi đọc qua 1 số bài báo về ngăn chặn và phát hiện Ddos/Botnet thì đa số đều sử dụng mô hình máy học hay những thuật toán máy học và có khá nhiều thuật nhưng nhóm chúng em chỉ sử dụng 4 thuật toán trong bài này. Sau khi thử nghiệm triển khai 1 số thuật toán máy học chống Ddos/Botnet trên bộ dữ liệu CTU-13, một bộ dữ liệu khá nổi tiếng trong việc thử nghiệm botnet thì thấy các thuật toán có độ chính xác sẽ khác nhau rất nhiều nhưng cũng có các thuật toán bị chênh lệch nhau rất nhiều, thuật toán Random Forest là thuật toán tốt nhất sau khi nhóm chúng em đã triển khai qua nhiều mô hình máy học thực nghiệm ở trên nền tảng google colab, thuật toán có sử dụng mã hóa One-hot Encoding, lựa chọn tính năng với chi-square và cân bằng bộ dữ liệu với NearMiss-1. Ngoài ra đồ án đã tổng hợp và giới thiệu đến mọi người: mạng botnet, cách chúng lây nhiễm, khả năng của chúng, xây dựng mô hình máy học thực nghiệm.

Hướng phát triển tiếp theo của nhóm với đề tài là sẽ thực hiện thử nghiệm mô hình máy học này trên môi trường máy ảo Vmware để kiểm tra tính chân thực và hiệu quả của mô hình khi chạy ngoài ra sẽ thử thêm với các thuật toán mới khác để tăng tính chính xác của mô hình máy học.

TÀI LIỆU THAM KHẢO

[1] Nguyễn Thành Hữu, "Nghiên cứu về DDOS và giải pháp ngăn chặn", Hà Nội, 2014.

[2]Gunter Ollmann, VP of Research, Damballa, Inc. , "Botnet Communication Topology, Understanding the intricacies of botnet Command-and-Control", 2009.

[3]. Jehad Ali, Rehanullah Khan, Nasir Ahmad, "Random Forests and Decision Trees", IJCSI International Journal of Computer Science Issues, Vol. 9, Issue 5, No 3, September 2012.

[4] Bentéjac, Candice&Csörgő, Anna&Martínez-Muñoz, Gonzalo. (2019). "A Comparative Analysis of XGBoost".

[5]. Chấn Phong. (2019). "*Botnet là gì? Cấu trúc và cách botnet hoạt động như thế nào?"*. Truy cập ngày 27/4/2022, từ https://quantrimang. com/botnet-hoat-dong-nhu-the-nao-36773#mcetoc_1dmsvgtdr0

[6]. NETACEA. "Types of botnets". Truy cập ngày 4/5/2022, từ https://www. netacea. com/glossary/types-of-botnets/

[7]. NETACEA. "*Historical List of botnets"*. Truy cập ngày 11/5/2022, từ https://www. netacea. com/glossary/list-of-botnets/

[8]. Attify. 2019. *"How Mirai Botnet Hijacks Your IoT Devices"*. Truy cập ngày 11/5/2022, từ https://www. attify. com/contact-us/

[9]. Catalin Cimpanu. (2020). *"Microsoft orchestrates coordinated takedown of Necurs botnet"*. Truy cập ngày 11/5/2022 từ https://www. zdnet. com/article/microsoft-orchestrates-coordinated-takedown-of-necurs-botnet/

[10]. Có chắc yêu là đây. (2019). *"IRC botnet hoạt động như thế nào?"*. Truy cập ngày 11/5/2022, từ https://quantrimang. com/botnet-hoat-dong-nhu-the-nao-phan-2-36793

[11]. TPhonghopamway. *"Học Có Giám Sát"* Truy cập ngày 17/5/2022, từ https://phonghopamway. com. vn/hoc-co-giam-sat/

[12]. Trí Tuệ Nhân Tạo. (2019). "*Cây Quyết Định (Decision Tree)"*. Truy cập 17/5/2022, từ https://trituenhantao. io/kien-thuc/decision-tree/

[13]. Nguyễn Thanh Bình. (2022). "*Các kỹ thuật phân lớp*". Truy cập ngày 17/5/2022, từ https://drive. google. com/file/d/1UBWgrAOy2kiR45SslbdrM4iy1MaYHheF/view

[14] Master's in Data Science. (2022). *"What Is Logistic Regression?"*. Truy cập ngày 17/5/2022, từ https://www. mastersindatascience. org/learning/introduction-to-machine-learning-algorithms/logistic-regression/

[15]. Wikipedia. (2022). *"Random forest"*. Truy cập ngày 17/5/2022, từ https://en. wikipedia. org/wiki/Random_forest#:~:text=Random%20forests%20or%20random%20decision, class%20selected%20by%20most%20trees.

[16]. Amanda Fawcett. (2021). *"Data Science in 5 Minutes: What is One Hot Encoding?"*. Truy cập ngày 7/6/2022, từ https://www. educative. io/blog/one-hot-encoding#what

[17]. Wikipedia. (2022). *"Chi-squared test"*. Truy cập ngày 7/6/2022, từ https://en. wikipedia. org/wiki/Chi-squared_test

[18]. Lê Thành Trung. (2021). "Xây Dựng Mô Hình Máy Học". Truy cập ngày 7/6/2022 https://github. com/Trung1808-code/KhoaLuanTotNghiep/blob/main/X%C3%A2y_d%E1%BB%B1ng_m%C3%B4_h%C3%ACnh_m%C3%A1y_h%E1%BB%8Dc. ipynb

[19]. Javatpoint. (2021). K-Nearest Neighbor(KNN) Algorithm for Machine Learning. Truy cập 18/5/2022, từ https://www. javatpoint. com/logistic-regression-in-machine-learning

[20]. BYJU'S. (2022). Anova Formula. Truy cập từ 7/6/2022, từ https://byjus. com/anova-formula/

[21] Aniruddha Bhandari. (2020). "Feature Scaling for Machine Learning: Understanding the Difference Between Normalization vs. Standardization". Truy cập 18/5/2022, từ https://www. analyticsvidhya. com/blog/2020/04/feature-scaling-machinelearning-normalization-standardization/#

[22] The CTU-13 dataset. (2011). A Labeled Dataset with Botnet, Normal and Background Traffic. Truy cập 21/6/2022, từ https://mcfp. weebly. com/the-ctu-13-dataset-a-labeled-dataset-with-botnet-normal-and-background-traffic. html